

Explainable Graph-Based Framework for Twitter Bot Detection

A PROJECT REPORT

Submitted by

ABDUL RAHIM S (220171601003)

AJAY M (220171601012)

Under the guidance of

Dr.S.Subhashini

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



B.S. Abdur Rahman[®]
Crescent
Institute of Science & Technology
Deemed to be University u/s 3 of the UGC Act, 1956

APRIL 2026



BONAFIDE CERTIFICATE

Certified that this project report “**Explainable Graph-Based Framework for Twitter Bot Detection**” is the bonafide work of **ABDUL RAHIM S (220171601003)** and **AJAY M (220171601012)** who carried out the project work under my supervision. Certified further, that to the best of our knowledge, the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr.S.Subhashini

Supervisor

Assistant Professor (Sr. Gr.)

Department of CSE

B S Abdur Rahman Crescent Institute of
Science and Technology Vandalur,
Chennai-600048.

SIGNATURE

Dr. W. AISHA BANU

Head of the Department

Professor

Department of CSE

B.S. Abdur Rahman. Crescent Institute
of Science & Technology Vandalur,
Chennai – 600048

VIVA VOICE EXAMINATION

The viva voce examination of the CSD 4251 - project work titled “**Explainable Graph-Based Framework for Twitter Bot Detection**”, submitted by **ABDUL RAHIM S (220171601003)** and **AJAY M (220171601012)** is held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We sincerely express our heartfelt gratitude to Prof. **Dr. A. PEER MOHAMED**, Vice chancellor I/C and **Dr. N. THAJUDDIN**, Pro-Vice Chancellor, B.S. Abdur Rahman Crescent Institute of Science and Technology, for providing us an environment to carry out our course successfully.

We sincerely thank **Dr. N. RAJA HUSSIAN**, Registrar for furnishing every essential facility for doing our project.

We thank **Dr. SHARMILA SANKAR**, Dean, School of Computer, Information and Mathematical Sciences for her motivation and support.

We thank **Dr. AISHA BANU W**, Professor and Head, Department of Computer Science and Engineering, for providing strong oversight of vision, strategic direction and valuable suggestions.

We express our sincere thanks to the Project Review Committee members, from the Department of Computer Science and Engineering, **DR. V. MUTHUPRIYA, Associate Professor** and **DR. MADHINA BANU, Associate Professor** for their valuable suggestions and support.

We obliged our project supervisor **Dr. S.SUBHASHINI, Assistant Professor (Sr. Gr.)**, Department of Computer Science and Engineering for her professional guidance and continued assistance during our project.

We thank our class advisor and project coordinator, **Dr. R. AKILA, Associate Professor** Department of Computer Science and Engineering for her guidance and encouragement throughout our project.

We thank all the **Faculty members** and the **System staff** of the Department of Computer Science and Engineering for their valuable support and assistance at various stages of project development. Sr

ABDUL RAHIM S

AJAY M

ABSTRACT

The system presents a multi-layered framework for detecting automated bot accounts on Twitter. The objective of the system is to accurately classify user accounts as human or bot while providing interpretable explanations for each prediction. Feature engineering and normalisation techniques are applied to user profile and activity-based attributes, reducing noise and producing a concise feature set that maintains detection accuracy across diverse datasets. By simplifying the data and eliminating unnecessary features through feature selection, the system processes information more efficiently without compromising performance. To capture relational context beyond individual accounts, GraphSAGE node embeddings are incorporated by retrieving the k-nearest neighbours from a training reference graph and averaging their embeddings, enabling the model to leverage social structure and interaction patterns. Model interpretability is achieved through LIME, which identifies and ranks the features most influential to each individual classification such as posting frequency, follower–following ratio, and engagement patterns providing transparent and actionable insights for researchers and platform administrators. Results are further communicated through an LLM-generated narrative summary, translating technical model outputs into plain-language explanations accessible to non-technical stakeholders.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iv
	LIST OF FIGURES	viii
	LIST OF TABLES	ix
	LIST OF ABBREVIATIONS	x
1	INTRODUCTION	1
1.1	OVERVIEW	1
1.2	DESCRIPTION	2
1.3	OBJECTIVE	2
1.4	ABOUT THE PROJECT	3
1.5	ORGANIZATION OF THE PROJECT	3
2	LITERATURE SURVEY	5
3	SYSTEM REQUIREMENTS AND DESIGN	8
3.1	PROBLEM DEFINITION	8
3.2	EXISTING SYSTEM	8
3.2.1	Key limitations of the existing system	9
3.3	PROPOSED SYSTEM	9
3.3.1	Objectives of the Proposed System	10
3.4	MODULE IDENTIFICATION	10
3.4.1	Data Collection and Preprocessing Module	10
3.4.2	Graph Construction Module	11
3.4.3	Classification Module	11

CHAPTER NO.	TITLE	PAGE NO.
3.4.4	Model Interpretation Module	12
3.4.5	Evaluation Metrics Module	12
3.5	SYSTEM ARCHITECTURE	13
3.6	USE CASE DIAGRAM	15
3.7	CLASS DIAGRAM	16
3.8	SEQUENCE DIAGRAM	17
3.9	SYSTEM REQUIREMENTS	18
3.10	DESIGN PROCESS AND EXPLANATION	18
3.11	PROPOSED ALGORITHM	24
4	SYSTEM METHODOLOGIES	24
4.1	MODULE EXPLANATIONS	24
4.1.1	Module 1: Data Collection and Processing	24
4.1.2	Module 2: Graph Construction and Representation Learning	26
4.1.3	Module 3: Classification and Explainability	27
4.1.4	Module 4: Evaluation and Validation	27
5	IMPLEMENTATION	29
5.1	MODULE IMPLEMENTATION	29
5.2	WORKFLOW DIAGRAM	34
5.3	DATASET	35
5.4	DATA PREPROCESSING	36

CHAPTER NO.	TITLE	PAGE NO.
5.5	FEATURE ENGINEERING	37
5.6	NORMALISATION AND SCALING	38
5.7	GRAPH EMBEDDING	39
5.8	MODEL TRAINING	40
5.9	LIME EXPLAINABILITY	42
5.10	LLM-BASED NARRATIVE GENERATION	43
5.11	WEB APPLICATION DEPLOYMENT	44
5.12	RESULT ANALYSIS	45
5.12.1	Output Analysis And Discussion	47
6	CONCLUSION AND FUTURE ENHANCEMENT	50
6.1	CONCLUSION	50
6.2	FUTURE ENHANCEMENTS	50
7	REFERENCES	52
8	APPENDIX	54
8.1	SOURCE CODE	54
8.2	PROJECT SCREENSHOTS	68
	TECHNICAL BIOGRAPHY	70

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.5.1	System Architecture Diagram	14
3.6.1	Use Case Diagram	15
3.7.1	Class Diagram	16
3.8.1	Sequence Diagram	17
5.1.1	Preprocessing Output	30
5.1.2	Engineered Features	31
5.1.3	Model Result	33
5.2.1	Workflow Diagram	34
5.3.1	Class Distribution	36
5.7.1	Bot Graph Representation	40
5.12.1	Feature Configuration Comparison	46
5.12.1.1	FF Ratio Comparison	49

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
3.9.1	Hardware Requirements	18
3.9.2	Software Requirements	18
5.1.1	Collected Data	29
5.1.2	Raw Uncleaned Data	29
5.1.3	Cleaned And Preprocessed Data	30
5.1.4	Similarity Graph Representation	31
5.1.5	GraphSAGE Training Loss Across Epochs	32
5.1.6	Node Embeddings	32
5.8.1	Feature Selection Process	41
5.12.1	Model Evaluation Metrics and Comparison	45
5.12.1.1	Feature Analysis	47

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the Curve
BERT	Bidirectional Encoder Representations from Transformers
CNN	Convolutional Neural Network
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
GCN	Graph Convolutional Network
GloVe	Global Vectors for Word Representation
GNN	Graph Neural Network
GraphSAGE	Graph Sample and Aggregation
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
k-NN	k-Nearest Neighbour
LIME	Local Interpretable Model-Agnostic Explanations
LLM	Large Language Model

Abbreviation	Full Form
LSTM	Long Short-Term Memory
ML	Machine Learning
NLP	Natural Language Processing
P(Bot)	Probability of Bot
P(Human)	Probability of Human
PKL	Pickle File (serialised model artefact)
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
SMOTE	Synthetic Minority Oversampling Technique
SVM	Support Vector Machine
URL	Uniform Resource Locator
XAI	Explainable Artificial Intelligence
XGBoost	Extreme Gradient Boosting
XLS	Excel Spreadsheet

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Social media platforms have evolved into powerful spaces for real-time communication, rapid information exchange, and large-scale public discourse, connecting hundreds of millions of users across the globe. Platforms like Twitter play a crucial role in shaping opinions, spreading news, and enabling conversations that transcend geographical boundaries. However, this openness and accessibility have also introduced significant vulnerabilities. One of the most concerning issues is the rise of automated accounts, commonly known as bots, which can be exploited for various forms of manipulation.

These bots exist on a spectrum. On one end, there are harmless automation tools designed to perform routine tasks such as posting updates or managing content. On the other end, there are highly sophisticated malicious bots created with the intent to spread misinformation, manipulate public opinion, artificially inflate engagement metrics, and coordinate large-scale inauthentic activities. Such bots can influence trending topics, amplify specific narratives, and even interfere with political or social discussions.

Over time, the sophistication of these bots has increased dramatically. Modern bots are no longer easily distinguishable from genuine users. They are capable of mimicking human-like behavior, including realistic posting frequencies, natural language usage, follower growth patterns, and interactive engagement with other users. Some advanced bots even participate in conversations in ways that appear authentic, making manual detection extremely difficult. This evolution has significantly raised the complexity of identifying and filtering out malicious accounts.

The presence of these deceptive entities poses serious risks to the integrity of the information ecosystem on platforms like Twitter. When users are exposed to manipulated content or artificially amplified opinions, it can distort their perception of reality and reduce trust in the platform. This erosion of trust not only affects individual users but also undermines the credibility of social media as a whole.

In conclusion, as bots continue to evolve in complexity and capability, developing intelligent and adaptive detection mechanisms remains a critical area of research. Ensuring the authenticity of user interactions and preserving the reliability of online information ecosystems is essential for maintaining trust in social media platforms.

1.2 DESCRIPTION

Social media platforms like Twitter have become increasingly vulnerable to automated bot accounts that manipulate public discourse, spread misinformation, and artificially influence trending topics. These bots have evolved far beyond simple scripts, now exhibiting coordinated behavior, adaptive posting strategies, and interaction patterns that closely resemble those of genuine users. Despite the growing severity and scale of this issue, many existing bot detection approaches rely heavily on complex machine learning or deep learning models that function as “black boxes.” While these models may achieve high accuracy, they often lack transparency in their decision-making process. This makes it difficult for moderators, researchers, and stakeholders to interpret why a particular account has been classified as a bot, reducing trust in the system and limiting its practical usability in real-world scenarios.

Furthermore, the absence of explainability poses challenges in critical applications such as content moderation, policy enforcement, and forensic analysis, where understanding the reasoning behind a decision is just as important as the decision itself. Without clear insights into model behavior, it becomes harder to validate results, identify biases, or improve detection strategies.

Therefore, there is a clear need for a robust, accurate, and interpretable bot detection system that not only identifies suspicious accounts effectively but also provides transparent, human-readable explanations for its classifications. Such a system would enhance trust, support better decision-making, and contribute to maintaining the integrity of social media platforms.

1.3 OBJECTIVE

The primary objective of this project is to design and implement an accurate and interpretable framework capable of automatically distinguishing between genuine human accounts and automated bot accounts on Twitter. The specific objectives are as follows:

- To develop a machine learning pipeline that analyses Twitter account metadata and behavioural activity patterns to reliably classify accounts as either human or bot, ensuring consistent detection performance across varied and imbalanced datasets.
- To enhance classification transparency by integrating an explainability layer that identifies the key behavioural signals driving each individual classification decision, and translates them into plain-language summaries through a large language model, making results accessible and actionable for non-technical users and platform administrators.

1.4 ABOUT THE PROJECT

The proposed system presents an end-to-end framework for detecting automated bot accounts on Twitter by analysing user profile metadata and behavioural activity patterns. At its core, the system processes raw account data through a structured pipeline that cleans, normalises, and engineers meaningful features reflecting indicators such as posting frequency, follower-following ratio, engagement metrics, and profile completeness. These features are then passed through a machine learning classifier that produces a probability score for each account, which is compared against an empirically derived threshold to assign a final verdict of either human or bot.

To go beyond individual account analysis, the system incorporates graph-based context by performing a similarity lookup over a reference graph of training accounts, retrieving the embeddings of the most similar accounts and averaging them to produce a relational representation for the input account. This allows the system to leverage social network structure and connectivity patterns alongside tabular features, capturing coordinated or community-driven bot behaviour that profile-level analysis alone would miss.

A central focus of the proposed system is interpretability. Rather than producing a black-box classification, the system applies an explainability module that identifies and ranks the specific behavioural features most responsible for each classification decision. These ranked signals are then passed to a large language model, which generates a concise, plain-language narrative summary of the reasoning behind the verdict, making the output transparent and understandable to non-technical stakeholders such as platform moderators and researchers. The complete system is accessible through an interactive web interface that enables on-demand account analysis, presenting the classification result, supporting evidence, and AI-generated explanation within a single unified view.

1.5 ORGANIZATION OF THE PROJECT

Chapter 1 - Introduction deals with an overview concerning the project that gives what it is about, why it is important, the major objectives, and how the report is structured

Chapter 2 - Literature Survey focuses on the existing research and previous work on the subject which helps to lay the base for the project and inform what has been done already.

Chapter 3 – System Requirements and Design defines both the problem as well as the scope of project failure in present-day solutions and the approach that is proposed. Furthermore, the system is divided into different modules and lists all the technical requirements.

Chapter 4 – System Methodologies dives into the core methods and algorithms used in each module of the project, explaining how the system works behind the scenes.

Chapter 5 – Implementation showcases how the system was actually built, with implementation details, screenshots, and an analysis of the results. This chapter also discusses any observations and challenges encountered along the way.

Chapter 6 – Result and analysis wraps up the project by summarizing the key findings and suggesting ways the system could be improved or expanded in the future.

CHAPTER 2

LITERATURE SURVEY

Innocent Mbona et al. [1] proposed a semi-supervised machine learning framework to classify social media bots into malicious and benign categories. Their approach focused on identifying behavioural irregularities in Twitter accounts by analysing features such as follower count, friend count, posting frequency, and interaction patterns. A key contribution of their work was the use of Benford's Law to detect anomalies in numerical distributions, which helped in identifying suspicious accounts that deviate from natural patterns. The study evaluated multiple models and found that a semi-supervised Support Vector Machine (SVM) performed best, demonstrating strong capability in distinguishing between different types of bots. Additionally, their work highlighted the importance of cybersecurity implications, as malicious bots are often involved in phishing, fake engagement, and misinformation campaigns.

Lynnette Hui Xian Ng et al. [2] conducted a large-scale analysis comparing bot and human behaviour across approximately 200 million Twitter users over multiple global events. Their research provided a fundamental understanding of how bots differ from humans in terms of communication patterns, linguistic behaviour, and network structure. Bots were observed to rely heavily on automated linguistic cues such as repeated hashtags, promotional content, and simplified language patterns, whereas humans demonstrated more context-aware and conversational language. In terms of network structure, bots typically formed star-shaped networks with centralized connections, while human users formed more complex and hierarchical social networks. This study also emphasized that bots can manipulate narratives, influence public opinion, and amplify misinformation at scale, making their detection critical for maintaining the integrity of online platforms.

Ryan Kenny et al. [3] investigated how effectively humans can detect social bots and explored methods to improve detection accuracy. Their study used experimental setups where participants were asked to classify Twitter accounts as bot or human. They tested three types of interventions: algorithm-generated bot scores, training videos, and warning messages. The results showed that providing bot scores and training significantly improved detection performance and reduced user overconfidence, while simple warnings had little impact. This research highlights that human detection alone is unreliable, and combining machine learning outputs with user guidance can enhance decision-making. It also emphasizes the importance of explainability in bot detection systems, as users are more likely to trust and correctly interpret model outputs when clear explanations are provided.

Abdullah Khan et al. [4] introduced a deep learning-based bot detection framework using the CrossGatedTabular architecture, designed to capture complex interactions between multiple features. Their model integrates user metadata, behavioural patterns, and textual features using gated neural networks, allowing it to learn high-dimensional representations automatically. To address the issue of class imbalance, they incorporated advanced techniques such as SMOTE-based sampling and regularization. The model achieved strong performance with high accuracy, precision, and recall, demonstrating the effectiveness of deep learning in capturing non-linear relationships in user behaviour. Their work also highlighted the limitations of traditional machine learning models, which rely heavily on manual feature engineering and struggle to adapt to evolving bot strategies.

Amine Sallah et al. [5] proposed a transformer-based bot detection model that leverages pre-trained language models such as BERT and GPT to analyze tweet content. Their approach focuses on generating contextual embeddings that capture semantic meaning, enabling more accurate classification of bot-generated text. The model uses a feedforward neural network on top of these embeddings to perform classification. Experimental results showed significant improvements in F1-score and accuracy compared to traditional approaches like Word2Vec and GloVe. Additionally, the study incorporated explainable AI techniques to improve transparency, demonstrating how feature importance and model reasoning can be interpreted. This work highlights the growing importance of NLP and transformer architectures in detecting sophisticated bots that generate human-like text.

Kadhim Hayawi et al. [6] presented a comprehensive survey of deep learning techniques for social media bot detection, focusing on models such as CNN, LSTM, BiLSTM, and graph neural networks. Their review showed that deep learning models consistently outperform traditional machine learning approaches due to their ability to automatically learn features from complex and high-dimensional data. The study categorized features into user metadata, content-based features, and network-based features, emphasizing that combining these feature types leads to better detection performance. It also highlighted key challenges such as the dynamic nature of bot behaviour, data imbalance, and lack of cross-platform generalization. The authors suggested that future systems should focus on adaptive and hybrid approaches to handle evolving threats.

Ana Aguilera et al. [7] developed CrediBot, a system that integrates bot detection into a broader credibility analysis framework for Twitter. Their approach combines text-based features, user profile attributes, and social impact metrics to evaluate both account authenticity and content reliability. They tested multiple machine learning algorithms and found that

Random Forest achieved the best performance, with accuracy and F1-score exceeding 97%. The study demonstrated that filtering out bot accounts significantly improves the credibility of information on social media platforms. It also highlighted the importance of combining bot detection with misinformation analysis to better understand the impact of automated accounts on online discourse.

Ashkan Dehghan et al. [8] focused on network-based bot detection using node and structural embeddings derived from social graphs. Their work compared classical embedding methods such as Node2Vec and DeepWalk with structural embedding techniques like Struc2Vec and Role2Vec. The results showed that structural embeddings, which capture the local neighbourhood patterns of users, provide better predictive performance than proximity-based embeddings. This indicates that bots often form distinct network structures that can be effectively captured using graph-based methods. Their study emphasizes the importance of incorporating network topology into bot detection systems, especially for identifying coordinated bot networks.

Zineb Ellaky et al. [9] proposed a hybrid deep learning architecture combining BiGRU and LSTM models with GloVe word embeddings for detecting bots based on textual content. Their model processes sequential tweet data to capture both semantic and temporal patterns in user behaviour. Experimental results showed extremely high accuracy and F1-scores on benchmark datasets such as Cresci-2017 and Twibot-20, demonstrating strong generalization capability. The study also highlighted that deep learning models are effective in handling unseen data and adapting to evolving bot behaviour. This reinforces the importance of using sequential models and semantic embeddings in modern bot detection systems.

Malak Aljabri et al. [10] provided a comprehensive literature review of machine learning-based bot detection techniques across multiple social media platforms, including Twitter, Facebook, Instagram, and LinkedIn. Their study categorized detection methods into supervised, semi-supervised, and unsupervised approaches, and analyzed the role of feature engineering in improving model performance. They highlighted that commonly used features include user profile attributes, content-based features, temporal activity patterns, and network-based features. The study also discussed key challenges such as scalability, evolving bot behaviour, and the need for cross-platform detection systems. It concluded that future research should focus on hybrid models that combine multiple data sources and adaptive learning techniques.

CHAPTER 3

SYSTEM REQUIREMENTS AND DESIGN

3.1 PROBLEM DEFINITION

The rapid growth of social media platforms such as Twitter (X) has significantly increased the presence of automated bot accounts that imitate human behaviour with a high degree of realism. These bots are frequently deployed to spread misinformation, shape public opinion, artificially boost the visibility of specific content, and distort engagement metrics such as likes, shares, and comments. In many cases, they operate not as isolated entities but as part of coordinated networks, where multiple accounts interact with each other to create the illusion of genuine activity. This coordinated behaviour makes the overall ecosystem more complex and misleading for users.

Detecting such accounts remains a challenging task because traditional approaches rely heavily on manually engineered features and basic profile-level analysis. While these methods were effective against earlier, simpler bots, they struggle to keep pace with the evolving strategies of modern automated systems. Advanced bots can replicate human-like posting patterns, maintain balanced follower–following ratios, and engage in conversations that appear natural. As a result, the boundary between real users and bots has become increasingly blurred, reducing the effectiveness of conventional detection techniques.

To address these limitations, there is a growing need for a robust and adaptive detection framework that goes beyond surface-level analysis. Such a system should incorporate both behavioural signals and network-based relationships to capture deeper patterns of interaction. By analysing how accounts behave over time and how they connect within a network, it becomes possible to identify both individual bots and coordinated bot groups more effectively. This integrated approach can significantly improve detection accuracy while adapting to the continuously evolving nature of bot activity.

3.2 EXISTING SYSTEM

The current method for spotting automated accounts relies heavily on Explainable Artificial Intelligence (XAI). This approach is used to find spambots and fake followers on platforms like Twitter (now known as X). Unlike older, "black box" machine learning models that make predictions without revealing their reasoning, XAI offers a clearer picture. It shows exactly

which account features or actions led to the bot classification. This level of clarity is crucial for social media companies, researchers, and regulatory agencies. They need detection results that are easy to interpret and provide justifiable detection results.

The system has been evaluated on benchmark datasets, particularly Cresci-15 and Cresci-17, using cross-validation to measure its performance. These datasets provide labeled examples of bot and human accounts, enabling supervised learning models to be trained and tested effectively.

3.2.1 Key Limitations of the Existing System

- Limited dataset diversity: Testing primarily on Cresci datasets restricts the system's exposure to varied bot behaviors and different social network structures. As a result, the system may underperform when deployed on newer datasets or platforms with diverse user populations.
- Generalization challenges: The model's performance tends to degrade on unseen datasets, particularly when bot behaviours evolve over time to avoid detection.
- High dependency on feature engineering: The system requires extensive manual selection and engineering of features, which is time-consuming, labour-intensive, and may not capture all relevant patterns in dynamic social networks.
- Adaptability issues: Rapidly evolving bot strategies, including AI-generated content and coordinated network behaviour, can limit the model's real-world effectiveness if updates are not made frequently.

Overall, while the existing system demonstrates the advantages of explainable AI in improving transparency and interpretability, it highlights the need for more generalizable, adaptable, and automated detection approaches capable of handling diverse datasets and evolving bot strategies.

3.3 PROPOSED SYSTEM

The proposed system develops a robust and interpretable framework for detecting bot accounts on Twitter by combining supervised machine learning, graph-based embeddings, and large language model-generated explanations. The framework analyses user metadata and activity-based features including account age, follower-following ratio, profile completeness, posting frequency, and engagement metrics, applying feature engineering and normalisation techniques to transform raw account data into structured numerical representations suitable for classification. To capture relational context beyond individual

accounts, GraphSAGE-based node embeddings are incorporated by performing a k-nearest neighbour lookup over a training reference graph, averaging the embeddings of the most similar accounts and appending them to the tabular feature vector. A Random Forest classifier is then trained on this combined representation, with an adaptive decision threshold derived from precision-recall analysis to maximise accuracy while minimising false positives across imbalanced datasets.

Interpretability is a central focus of the proposed framework. LIME is applied at the individual account level to identify and rank the behavioural features most influential to each classification decision, ensuring that every verdict is supported by transparent, auditable evidence rather than a black-box output. These LIME explanations are subsequently passed to a large language model, which generates a polished plain-language narrative summary for each account, making classification outcomes accessible and understandable to non-technical users and platform administrators. The complete system is deployed as an interactive web application, enabling on-demand account analysis with the classification verdict, LIME evidence, and AI-generated explanation presented together in a single unified interface.

3.3.1 Objectives of the Proposed System

- Hybrid feature representation combines tabular behavioral features with GraphSAGE embeddings to capture both individual traits and network patterns for stronger detection.
- Per-account explainability uses LIME to highlight key signals behind each prediction, ensuring transparent and auditable decisions.
- Plain-language summaries use LLM-generated reports to convert LIME insights into clear explanations for non-technical users.
- Interactive web deployment provides a browser-based interface for real-time analysis with predictions, evidence, and explanations in one view.

3.4 MODULE IDENTIFICATION

3.4.1 Data collection and preprocessing module

The data collection and preprocessing stage forms the foundation of the bot detection system by ensuring that diverse datasets are properly integrated and standardized. In this work, the Cresci-17 and Twibot-22 datasets were combined, bringing together user profiles, content attributes, and interaction-level information into a unified schema. This integration enables

consistency across different data sources and supports future extensibility. Careful alignment of fields ensures that variations between datasets do not affect downstream analysis, allowing the model to learn from a broader and more representative set of user behaviors.

To improve the quality of input data, extensive preprocessing and feature engineering were applied. Meaningful behavioural, profile-based, and activity-related features were derived, along with temporal patterns that capture how user activity evolves over time. Missing values and noisy entries were handled systematically, and normalization techniques were applied to maintain consistency across features. In addition, text data was processed through tokenization and stopword removal, followed by preparation for embedding generation, enabling the model to capture both structural and semantic aspects of user activity more effectively.

3.4.2 Graph Construction Module

A similarity-based user graph is constructed to capture relationships between accounts by identifying users with comparable behaviour patterns. This is achieved using k-Nearest Neighbors (k-NN) combined with cosine similarity on normalized feature vectors, ensuring that users with similar activity, profile, and engagement characteristics are closely connected. By transforming tabular data into a graph structure, the model is able to represent not only individual user attributes but also the contextual relationships between users, which is particularly useful for identifying coordinated or group-based bot activity.

To effectively leverage this graph structure, the GraphSAGE algorithm is applied to learn node embeddings that incorporate both a user's own features and information from its neighboring nodes. This process enables the generation of rich, graph-informed representations that reflect local network patterns and behavioural similarities. These embeddings are then used as enhanced input features for downstream bot classification, improving the model's ability to detect both isolated bots and those operating within coordinated networks.

3.4.3 Classification Module

The classification and inference stage focuses on accurately distinguishing between bot and human accounts using a Random Forest model trained on a combination of engineered features and graph-based similarity representations. By integrating both behavioural attributes and graph-derived embeddings, the model captures both individual user patterns and relational context within the network. This enriched feature space improves the model's ability

to detect subtle and complex bot behaviours, including those that closely resemble legitimate user activity.

Beyond prediction, the system also incorporates model optimization and interpretability mechanisms to enhance reliability and trust. Techniques such as feature selection, class imbalance handling, and hyperparameter tuning are applied to improve classification performance. Additionally, explainability methods are integrated to provide human-readable justifications for each prediction, enabling better understanding of the model's decisions. This not only supports validation and debugging but also makes the system more suitable for real-world deployment where transparency is essential.

3.4.4 Model Interpretation Module

To enhance interpretability, the system applies LIME (Local Interpretable Model-Agnostic Explanations) to analyze individual predictions and identify the contribution of each feature to the final classification. Instead of treating the model as a black box, LIME approximates the model locally around a specific prediction and highlights which features influenced the decision towards “bot” or “human.” This enables a detailed, instance-level understanding of model behavior, making it easier to interpret why a particular account was flagged.

Building on this, a Large Language Model (LLM) is used to transform the technical LIME outputs into concise, human-readable explanations. The LLM summarizes key contributing factors and presents them in a clear and structured manner that can be easily understood by non-technical users. This combination of LIME and LLM not only improves transparency but also bridges the gap between complex model reasoning and practical usability, making the system more accessible for moderators, analysts, and decision-makers.

3.4.5 Evaluation Metrics Module

The performance of the proposed system is evaluated using standard classification metrics including accuracy, precision, recall, F1-score, and Area Under the Curve (AUC). These metrics provide a comprehensive assessment of the model's effectiveness from multiple perspectives. While accuracy gives an overall measure of correctness, precision and recall offer deeper insight into how well the model distinguishes between bots and humans, particularly in handling false positives and false negatives. The F1-score balances these two aspects, and AUC evaluates the model's ability to separate classes across different threshold settings, ensuring a more robust evaluation.

To further validate the model's generalization capability, cross-dataset testing is performed between Cresci and Twibot datasets. This involves training the model on one dataset and testing it on the other, ensuring that the system is not overfitting to dataset-specific patterns. Such evaluation demonstrates the model's ability to perform effectively across different data distributions and real-world scenarios, which is critical for practical deployment.

3.5 SYSTEM ARCHITECTURE

The proposed system architecture begins with a Twitter user ID being submitted through the user interface, which acts as the entry point of the system. Once the user provides the ID, the interface retrieves the corresponding data from the stored dataset (Excel file) and passes it forward for processing. This initial step ensures that the system can access all required user profile information, activity details, and interaction-related attributes in a structured format. The retrieved data is then forwarded to the feature extraction stage, where relevant behavioural and profile-based features are identified and organized for further processing.

Following feature extraction, the data undergoes preprocessing and normalization to improve its quality and consistency. During this stage, raw values are cleaned, scaled, and transformed into a structured numerical format that can be effectively used by machine learning models. This ensures that variations in data scale or inconsistencies do not negatively impact model performance. At the same time, the processed features are used to construct a similarity graph, where users with similar behavioural patterns are connected using techniques such as k-Nearest Neighbors and cosine similarity. This graph represents the relationships between users and captures important structural information that cannot be obtained from individual features alone.

The similarity graph is then passed to the Graph Neural Network (GraphSAGE), which learns node embeddings for each user. These embeddings encode both the user's individual characteristics and the influence of neighbouring users in the graph, providing a richer representation of user behaviour. The generated node embeddings are then combined with the preprocessed feature set and passed to the classifier model. The classifier, based on a Random Forest algorithm, analyzes this combined input and produces a prediction indicating whether the account is a bot or a human.

To enhance transparency and interpretability, the prediction is further processed using LIME, which identifies the most important features that influenced the classification decision. LIME provides a local explanation by highlighting how each feature contributed to the final outcome. These feature-level explanations are then passed to a large language model, which converts

them into a clear and human-readable summary. This step ensures that the explanation is not only technically accurate but also easy to understand for non-technical users.

Finally, the system presents the output to the user in the form of a bot or human label along with an interpretable explanation of the reasoning behind the prediction. This end-to-end architecture ensures a smooth flow from input data to final output, combining accurate classification with meaningful explanations, and making the system both effective and user-friendly.

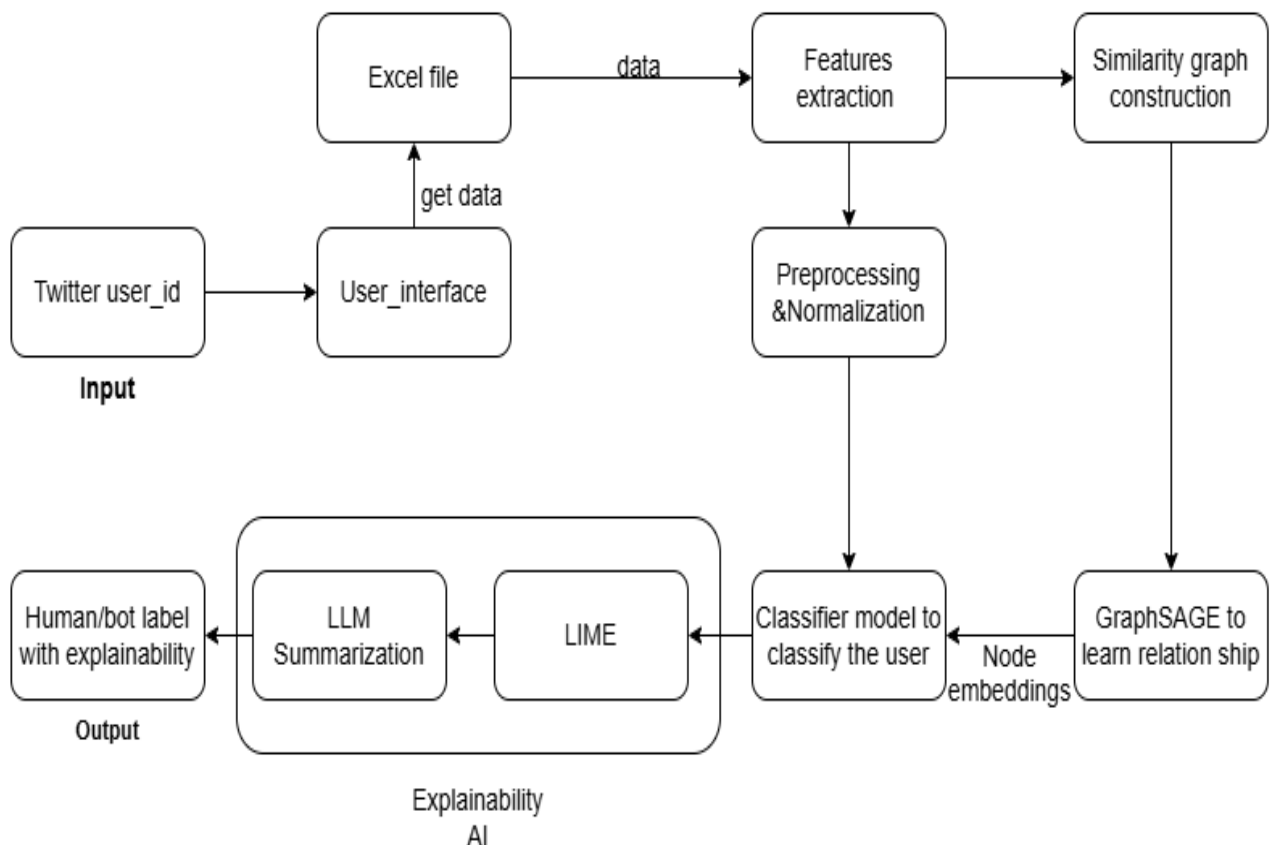


Figure 3.5.1 System Architecture Diagram

The figure 3.5.1 illustrates the bot detection workflow, from data preprocessing and graph construction using Cresci-17 and Twibot-20, through feature extraction and detection models (GraphSAGE, Relational GCN, XGBoost, Random Forest), to evaluation metrics ,explainability analysis , and LLM Summarization and finally the classification of human or bot.

3.6 USE CASE DIAGRAM

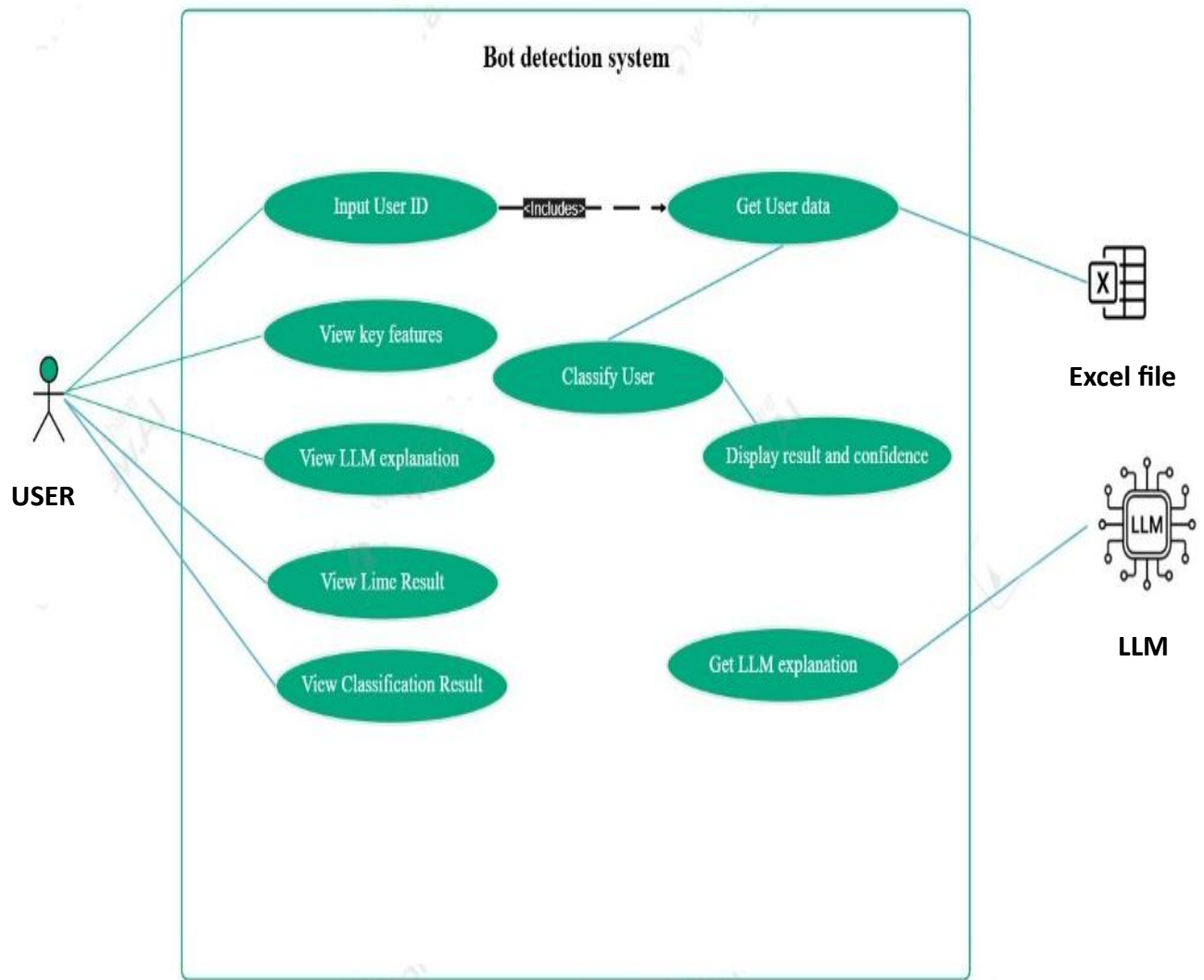


Figure 3.6.1 Use Case Diagram

The figure 3.6.1 illustrates the use case diagram of the bot detection system, showing the interaction between the user and system components for data retrieval, classification, and explanation generation.

This diagram represents the overall workflow of the bot detection system from a user interaction perspective. The user begins by entering a user ID, after which the system retrieves the corresponding user data and performs classification using the trained model. The system then displays the classification result along with a confidence score. Additionally, the user can view key features, LIME-based explanations, and LLM-generated summaries to understand the reasoning behind the prediction. The diagram highlights how different components, including data processing, classification, and explainability modules, work together to provide both accurate and interpretable results.

3.7 CLASS DIAGRAM

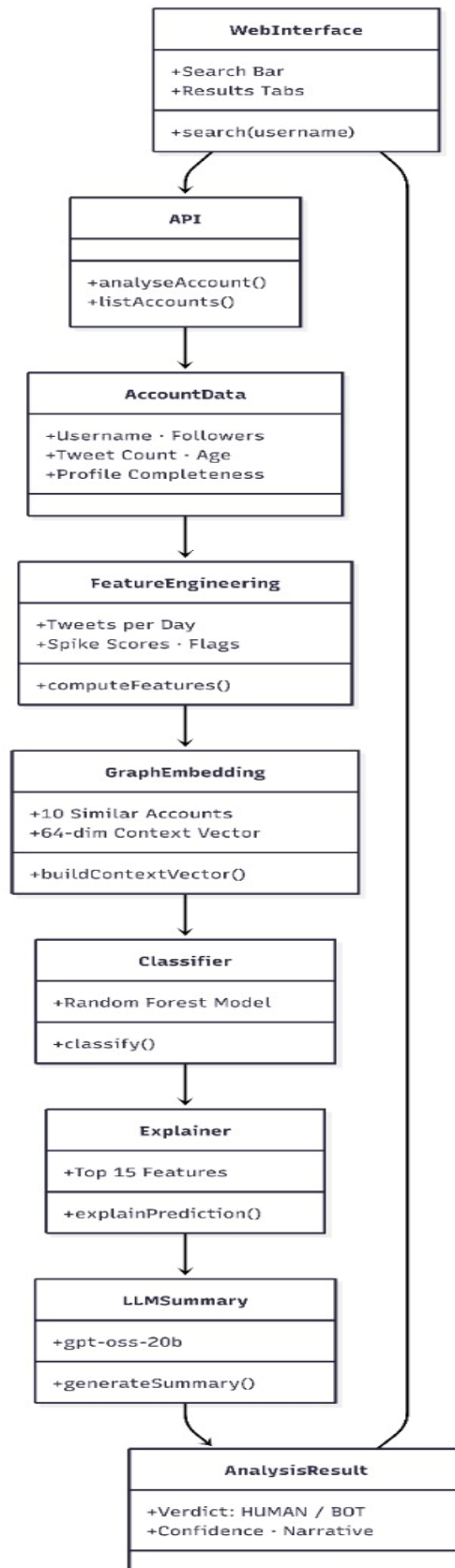


Figure 3.7.1 Class Diagram

The figure 3.7.1 illustrates the structural organization of the bot detection system and the relationships between its core components. The WebInterface interacts with the API to handle user queries and retrieve account data. The AccountData class stores user-related attributes such as followers, tweets, and profile details, which are then processed by the FeatureEngineering module to derive behavioural features. The GraphEmbedding component generates contextual representations using similar accounts, which are combined with engineered features and passed to the Classifier (Random Forest) for prediction. The Explainer module identifies the most influential features, and the LLMSummary component converts these into a human-readable explanation. Finally, the AnalysisResult class presents the classification outcome along with confidence and explanation.

3.8 SEQUENCE DIAGRAM

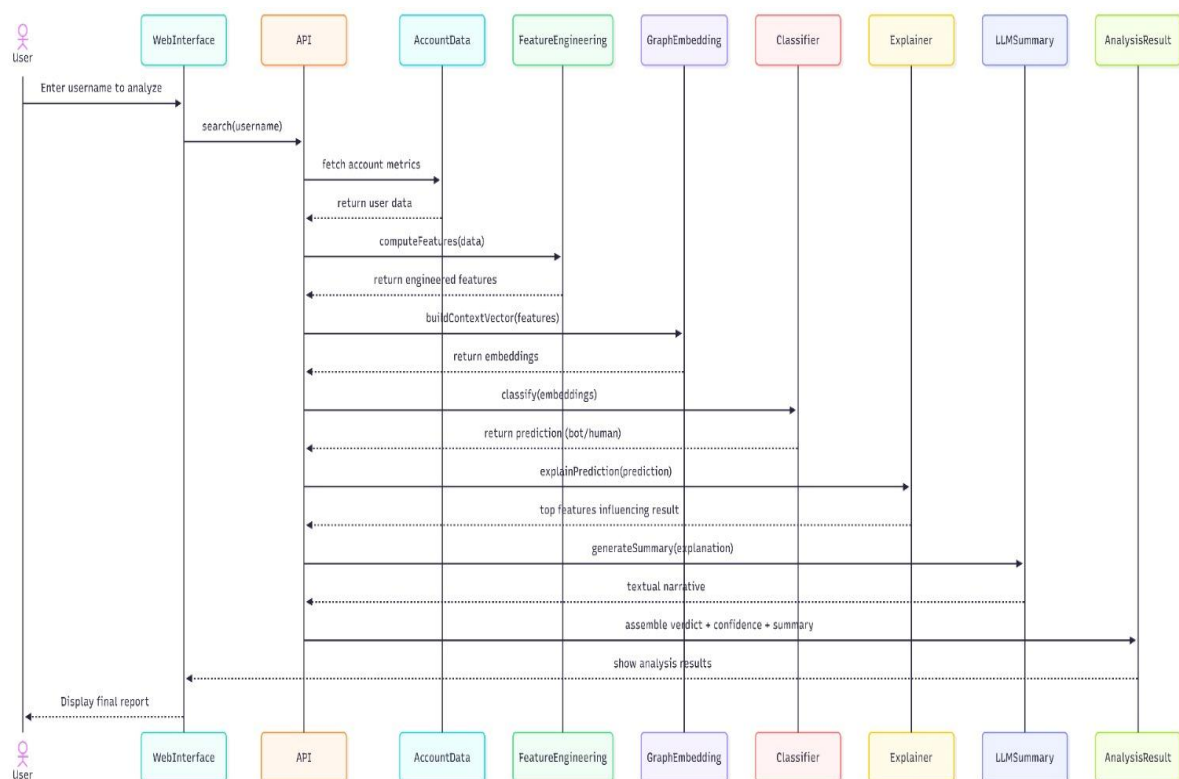


Figure 3.8.1 Sequence Diagram

The figure 3.8.1 illustrates the interaction between system components when a user submits a username for analysis. The request flows from the Web Interface through the API to retrieve account data, which is then processed for feature extraction and graph embedding. The classifier generates a bot or human prediction, followed by explanation using LIME and summarization through the LLM. Finally, the system compiles the prediction, confidence, and explanation into a result that is returned to the user interface for display.

3.9 SYSTEM REQUIREMENTS

Table 3.9.1 : Hardware Requirements

Hardware	Specifications
Processor	Intel i5 or AMD Ryzen 5 Series
RAM	Minimum 8 GB RAM Recommended
Storage	Atleast 500 MB storage Required
GPU	NVIDIA GTX 1650

The table 3.9.1 illustrates the hardware specifications outline information regarding the required hardware or PC components that are minimum requirement to run the framework.

Table 3.9.2 : Software Requirements

Software	Specification
Python	Python version 3.8+
Operating System	Windows, Linux or MacOS
Visual Studio Code	VS Code Version 1.85+
Open router api	openai/gpt-oss-20b:free

The table 3.9.2 illustrates the software specifications define the necessary tools, framework, and libraries required.

3.10 DESIGN PROCESS AND EXPLANATION

The Social Media Bot Detection and Explainability System was designed to provide an accurate, scalable, and interpretable solution for identifying automated accounts on platforms such as Twitter (X). With the rapid growth of social media and the increasing sophistication of bot behaviour, it has become essential not only to detect bots effectively but also to understand and explain the reasoning behind each prediction. The primary objective of this system is to combine traditional machine learning techniques with graph-based representations to capture both individual user behaviour and network-level interactions. At the same time, the system emphasizes explainability, ensuring that every classification decision can be clearly interpreted by users, analysts, and moderators. Each stage of the system design is carefully structured to balance detection accuracy, computational efficiency, interpretability, and practical usability in real-world environments.

1. Requirement Analysis

The initial stage of the design process focused on understanding the core problem of bot activity in social media environments and defining clear, achievable system objectives. Social media platforms face challenges due to the presence of both isolated bots and coordinated bot networks that manipulate engagement and spread misinformation. Therefore, the system was designed to detect both individual bot accounts and groups of coordinated bots by analyzing behavioural patterns and network relationships. In addition, a key requirement was to ensure transparency in decision-making, enabling users to understand why a particular account was classified as a bot or human.

Functional Requirements

- The system should process user profile, activity, and interaction data from multiple datasets by collecting and integrating information such as follower counts, tweet activity, and engagement patterns into a structured format.
- It should extract meaningful features representing user behaviour and engagement by transforming raw data into useful indicators such as activity rates, interaction patterns, and profile completeness.
- It should classify accounts as bot or human using machine learning models by learning patterns from both behavioural features and graph-based representations to improve detection accuracy.
- It should provide explainable outputs for each prediction using interpretability techniques by highlighting key features and generating human-readable explanations that clearly justify the classification result.

Non-Functional Requirements

The system must handle large-scale social media data efficiently by processing large volumes of user data without significant delays or performance issues.

- The architecture should be modular for easy maintenance and upgrades, allowing new features, datasets, or models to be added without affecting the overall system structure.
- The system should be reliable, interpretable, and suitable for real-world deployment by producing consistent results and ensuring that predictions can be easily understood by both technical and non-technical users.

2. System Architecture: How Everything Works Together

The system follows a multi-layered architecture, where each layer performs a specific function and contributes to the overall bot detection pipeline. This layered design ensures that the flow of data is organized and efficient, starting from raw data collection and progressing through processing, modeling, and explanation. Each component is designed to work independently yet seamlessly with others, making the system scalable, modular, and easy to maintain.

i. Data Collection Layer

This layer is responsible for gathering and integrating data from multiple sources such as the Cresci-17 and Twibot-22 datasets. It combines user profile information, content-related features, and interaction data into a unified and consistent format. By aligning different datasets into a common schema, this layer ensures that variations in data structure do not affect further processing. This step provides a comprehensive and diverse dataset that improves the system's ability to learn different types of user behaviour.

ii. Data Processing Layer

In this layer, the collected raw data is cleaned and prepared for analysis. Missing values, inconsistencies, and noisy entries are handled to improve data quality. The data is then normalized to ensure uniformity across features. Feature engineering is performed to extract meaningful behavioural, activity-based, and profile-related attributes such as engagement patterns, posting frequency, and account characteristics. This transformation converts raw data into structured inputs that are suitable for machine learning models.

iii. Graph and Embedding Layer

This layer focuses on capturing relationships between users by converting the processed data into a graph structure. A similarity-based graph is constructed using k-Nearest Neighbors (k-NN) with cosine similarity, where users with similar behaviour are connected. GraphSAGE is then applied to learn node embeddings, which combine information from a user and its neighbouring nodes. These embeddings capture both individual behaviour and network-level patterns, making them highly effective for identifying coordinated bot activity.

iv. Classification Layer

The classification layer uses a Random Forest model to distinguish between bot and human accounts. It takes both engineered features and graph-based embeddings as input, allowing the model to consider both individual and relational characteristics. By combining these features, the model can detect complex patterns and subtle differences between bots and

genuine users. This improves classification accuracy, especially for advanced bots that attempt to mimic human behaviour.

v. Explainability Layer

The final layer focuses on making the model's predictions understandable. LIME is used to identify which features contributed most to a particular classification by analyzing the model locally around each prediction. These technical explanations are then passed to a Large Language Model (LLM), which converts them into clear, human-readable summaries. This ensures that users can easily understand why an account was classified as a bot or human, improving transparency and trust in the system.

3. Data Preprocessing Pipeline: Preparing Social Media Data

Before training the model, the collected data is carefully prepared to ensure it is clean, consistent, and suitable for analysis. Raw social media data often contains noise, missing values, and inconsistencies that can negatively affect model performance if not handled properly. Therefore, a structured preprocessing pipeline is applied to improve data quality and transform it into a format that the model can effectively learn from. This stage plays a crucial role in ensuring that meaningful patterns can be extracted from the data.

The preprocessing steps include:

- **Data Cleaning:** Unnecessary noise, duplicate records, and inconsistent entries are removed to ensure that the dataset is accurate and reliable. This helps prevent misleading patterns during model training.
- **Handling Missing Values:** Missing or incomplete data is addressed using appropriate statistical methods such as filling with median or default values, ensuring that no important records are lost.
- **Feature Engineering:** New features are created from existing data to better represent user behaviour. These include activity-based indicators such as posting frequency, engagement levels, and interaction patterns, which provide deeper insights into how users behave.
- **Normalization:** Numerical features are scaled to a consistent range so that no single feature dominates the model due to differences in magnitude. This improves the stability and performance of machine learning algorithms.
- **Text Processing:** Textual data is processed using techniques such as tokenization and stopword removal, followed by preparation for embedding generation. This allows the system to capture semantic information from user-generated content.

Overall, these preprocessing steps ensure that both structured data and textual information are properly refined and transformed, making them suitable for machine learning models and graph-based analysis.

4. Model Training and Evaluation: Learning Bot Behaviour

After completing the preprocessing stage, the refined dataset is used to train machine learning models capable of distinguishing between bot and human accounts. At this stage, the model learns patterns from the data by analyzing user behaviour, activity levels, profile characteristics, and network relationships. By combining both engineered features and graph-based representations, the system is able to capture complex patterns that go beyond simple rule-based detection. This allows the model to identify even sophisticated bots that attempt to imitate genuine user behaviour.

To measure how well the model performs, several standard classification metrics are used. These metrics provide a detailed understanding of the model's strengths and weaknesses from different perspectives:

- **Accuracy** – Indicates the overall percentage of correct predictions made by the model.
- **Precision** – Measures how many of the accounts predicted as bots are actually bots, helping to reduce false positives.
- **Recall** – Evaluates how well the model identifies all actual bot accounts, ensuring fewer bots are missed.
- **F1-Score** – Provides a balanced measure by combining precision and recall into a single value.
- **AUC (Area Under Curve)** – Reflects the model's ability to distinguish between bot and human classes across different thresholds.

5. Integration and Prediction: End-to-End Execution

After the model has been trained and validated, all individual components are combined into a single, unified pipeline to perform predictions in a smooth and efficient manner. This integration ensures that the system can take raw input data and process it step by step without any manual intervention. The goal of this stage is to create a seamless flow where each module works together to produce not only an accurate classification but also a meaningful explanation for the result.

The system follows a structured sequence of operations:

- The input user data is first preprocessed, where it is cleaned, normalized, and transformed into meaningful features that represent user behaviour and activity.
- Based on these processed features, graph-based embeddings are generated using similarity relationships between users. This step helps capture both individual characteristics and network-level patterns.
- The combined feature set is then passed to the Random Forest model, which predicts whether the account is a bot or a human along with a confidence score.
- To make the prediction understandable, LIME is applied to identify the most influential features that contributed to the decision.
- Finally, a Large Language Model (LLM) converts these technical feature contributions into a clear and easy-to-understand explanation for the user.

This end-to-end pipeline ensures a smooth transition from raw data to final output, providing both accurate classification and transparent explanations. It makes the system practical for real-world use, where users not only need predictions but also a clear understanding of how those decisions are made.

6. Continuous Feedback and Improvements

The system is designed with a modular structure, which makes it easy to update and improve individual components without affecting the entire pipeline. As new datasets become available or new patterns of bot behaviour emerge, they can be incorporated into the system with minimal effort. Similarly, improvements in feature engineering techniques or the adoption of more advanced models can be integrated smoothly, allowing the system to evolve over time without requiring a complete redesign.

This flexibility plays a key role in maintaining the long-term effectiveness of the bot detection system. Since social media environments are constantly changing and bots continue to become more sophisticated, the system must be able to adapt accordingly. By supporting continuous updates and improvements, the system remains reliable, scalable, and relevant, ensuring that it can effectively handle new challenges and provide accurate results in dynamic real-world scenarios.

3.11 Proposed Algorithm:

Input:

User ID

Output:

Account classification Whether Human or Bot with Explanation.

Step 1 : Start

Step 2 : Get user id and extract twitter raw profile metadata and activity based attributes.

Step 3 : Preprocess input data, Clean the data, handle missing values.

Step 4 : Perform feature engineering to compute behavioral attributes.

Step 5 : Identify similar accounts using K-Nearest Neighbors based on feature similarity

Step 6 : Generate graph-based embedding by aggregating precomputed GraphSAGE embeddings of similar accounts.

Step 7 : Combine traditional machine learning features with the generated graph embeddings to form a hybrid feature set.

Step 8 : Apply scaling and normalization using pre-trained scalers.

Step 9 : Select required features based on pre-trained feature list.

Step 10 : Apply the trained model to classify accounts as bot or human using probability thresholds.

Step 11 : Apply the LIME explainability method to identify the most influential features.

Step 12 : Generate natural language explanation using LLM

Step 13 : Display classification result and explanation

Step 14 : Stop

CHAPTER 4

SYSTEM METHODOLOGIES

4.1 MODULE EXPLANATIONS

4.1.1 Module 1: Data Collection and Preprocessing

1. Data Integration and Preparation

Input: Raw user profile, content, and interaction data from Cresci-17 and Twibot-22 datasets

Output: Unified and structured dataset with aligned features across sources

Method:

- Data from multiple datasets is merged into a common schema to ensure consistency across different sources.
- User identifiers are standardized, and dataset fields are aligned to maintain uniformity and avoid inconsistencies during further processing.

Description:

This module forms the base of the system by combining multiple datasets into a single structured format. By aligning user profiles, content features, and interaction data, the system ensures that variations between datasets do not affect downstream processing. This integration enables the model to learn from a wider and more diverse set of user behaviours, improving its ability to generalize across different scenarios.

2. Data Cleaning and Feature Engineering

Input: Integrated raw dataset

Output: Cleaned, normalized dataset with engineered behavioural and activity-based features

Method:

- Missing values and noisy entries are handled using statistical imputation and validation techniques.
- Feature engineering is applied to derive behavioural indicators, and normalization techniques are used to scale numerical features consistently.
- Text data is processed through tokenization and stopword removal, followed by preparation for embedding generation.

Description:

This stage enhances the quality and usefulness of the data by transforming raw inputs into

meaningful features. Behavioural, profile-based, and activity-related attributes are extracted, along with temporal patterns that reflect user activity over time. These transformations ensure that both structured and textual data are suitable for machine learning, allowing the model to capture deeper patterns in user behaviour.

4.1.2 Module 2: Graph Construction and Representation Learning

1. Similarity-Based Graph Construction

Input: Normalized user feature vectors

Output: User similarity graph representing relationships between accounts

Method:

- k-Nearest Neighbors (k-NN) is applied using cosine similarity to identify similar users based on feature vectors.
- Each user is connected to its closest neighbours, forming a graph structure that reflects behavioural similarity.

Description:

This module converts tabular data into a graph format, enabling the system to capture relationships between users. By linking accounts with similar activity and profile characteristics, the graph highlights patterns that may indicate coordinated behaviour. This representation is especially useful for detecting bot networks that operate collectively rather than individually.

2. Graph Embedding using GraphSAGE

Input: User graph and feature data

Output: Low-dimensional node embeddings representing user behaviour and network context

Method:

- GraphSAGE aggregates information from neighbouring nodes to learn meaningful embeddings for each user.
- The learned embeddings combine both individual features and neighbourhood information.

Description:

GraphSAGE generates vector representations that capture both local structure and behavioural similarity within the network. These embeddings provide a richer representation of users compared to standalone features, enabling the system to detect subtle patterns and coordinated bot activity more effectively.

4.1.3 Module 3: Classification and Explainability

1. Bot Classification using Random Forest

Input: Combined feature set of engineered attributes and graph embeddings

Output: Classification label (bot/human) with associated prediction score

Method:

- A Random Forest model is trained using both behavioural features and graph-based embeddings.
- Feature selection, class balancing, and hyperparameter tuning are applied to improve model performance.

Description:

This module performs the core task of distinguishing between bot and human accounts. By combining individual behavioural features with graph-derived representations, the model captures both user-level and network-level patterns. This improves detection accuracy, especially for advanced bots that closely mimic human behaviour.

2. Model Interpretation using LIME and LLM

Input: Model prediction and feature vector

Output: Ranked feature contributions and human-readable explanation

Method:

- LIME is used to analyze feature-level contributions for each prediction by approximating the model locally.
- A Large Language Model (LLM) converts the technical output into a structured, easy-to-understand explanation.

Description:

This component improves transparency by explaining why a specific account was classified as a bot or human. LIME identifies the most influential features, while the LLM translates these insights into clear, natural language summaries. This makes the system more interpretable and accessible for users, moderators, and analysts.

4.1.4 Module 4: Evaluation and Validation

1. Performance Evaluation Metrics

Input: Model predictions and ground truth labels

Output: Quantitative performance scores

Method:

- Evaluation is performed using accuracy, precision, recall, F1-score, and AUC metrics.
- Each metric provides insight into different aspects of model performance, including error handling and class separation.

Description:

This module ensures that the model performs reliably across multiple evaluation criteria. By using a combination of metrics, the system provides a balanced assessment of its ability to detect bots while minimizing incorrect classifications.

2. Robustness Testing

Input: Trained model and unseen data

Output: Performance consistency on new unseen data

Method:

- The model is evaluated on unseen data to measure its ability to handle new patterns.
- Performance is analyzed to assess stability against evolving bot behaviours.

Description:

This stage validates the system's robustness in real-world scenarios. By testing the model on unseen data, it ensures that the model does not rely only on learned patterns from training data but can adapt to new and changing bot behaviours. This helps confirm that the system remains reliable and effective even as bot strategies evolve over time.

CHAPTER 5

IMPLEMENTATION

5.1 MODULE IMPLEMENTATION

1. Data Integration and Preparation

verified	friends_count	followers_count	listed_count	Label	tweets_count	hashtag_count
	287	222	1	bot	110	96
	366	290	1	bot	116	102
	0	0	0	bot	66	75
	4685	4548	2	bot	291	81
	5251	4879	4	bot	294	94

Table 5.1.1 Collected Data

The table 5.1.1 illustrates the sample view of the integrated dataset after combining data from multiple sources such as Cresci-17 and Twibot-22. The table includes key user-level attributes such as user_name, verified status, friends count, followers count, listed count, tweet activity metrics, and the label indicating whether the account is a bot or human. This figure illustrates how raw profile, content, and interaction features are aligned into a unified and structured format. Such integration ensures consistency across datasets and prepares the data for further preprocessing and feature engineering stages in the bot detection pipeline.

2. Data Cleaning Feature Engineering

verified	friends_count	followers_count	listed_count	Label	tweets_count	hashtag_count
	0	0	0	bot	150	84
	5484	5149	6	bot	302	106
	4605	4208	2	bot	259	97
	0	0	0	bot	121	82
	5272	4803	5	bot	309	84

Table 5.1.2 Raw Uncleaned Data

The table 5.1.2 illustrates the sample of the raw, uncleaned dataset collected from multiple sources before preprocessing. The data contains user profile attributes such as user ID,

username, verification status, follower and friend counts, tweet statistics, and labels. At this stage, the dataset may include inconsistencies, missing values, and noise, which require further cleaning and standardization before it can be used for analysis.

verified	friends_count	followers_count	listed_count	Label	tweets_count	hashtag_count
0	322	374	4	1	851	3
0	2976	159	2	1	845	783
0	2	4745	106	1	842	10
0	1273	488	4	1	840	0
0	2791	238	0	1	839	7

Table 5.1.3 Cleaned And Preprocessed Data

The table 5.1.3 illustrates the cleaned and preprocessed dataset after applying data cleaning techniques. In this stage, missing values are handled, inconsistencies are corrected, and features are normalized to ensure uniformity across all records. The dataset is structured and refined, making it suitable for further feature extraction and machine learning processes.

```

Original shape: (19999, 34)
Original label distribution:
Label
human    13968
bot       6031
Name: count, dtype: int64

User ID length check:
count    19999.000000
mean      10.761788
std       3.586062
min        6.000000
25%        9.000000
50%       10.000000
75%       10.000000
max       19.000000
Name: user_id, dtype: float64

Unknown labels: 0

Final shape: (19997, 34)

Final label distribution:
Label
0      13966
1       6031
Name: count, dtype: int64

```

Figure 5.1.1 Preprocessing Output

The figure 5.1.1 illustrates the output of the preprocessing script, highlighting key dataset statistics such as original and final shape, label distribution, and validation checks. It also confirms successful cleaning operations, including removal of invalid entries, normalization of labels, and preservation of user ID integrity. This output ensures that the dataset is correctly prepared before moving to the next stage.

3. Feature Engineering

AI	AJ	AK	AL	AM	AN	AO	AP	AQ	AR	AS
tweets_per_day	followers_per_day	log_tweets_per_day	log_followers_per_day	followers_spike	tweet_spike	extreme_activity_score	young_account_flag	huge_followers_flag	young_and_popular	
4183.185467	1112.294118	8.339067332	7.015078573	42281.80939	159016.08	15.35414591	0	1	0	
1159.175028	14.82218506	7.056326159	2.761413073	627.8030925	49097.5969	9.817739232	0	0	0	
19080.29032	62.95698925	9.85646357	4.158210814	607.1352173	184003.656	14.01467438	0	0	0	
1222.9729	18.42547425	7.109857322	2.966585311	613.0451846	40690.2768	10.07644263	0	0	0	
990.4438378	2.848673947	6.899162303	1.34772866	101.9968492	35462.8689	8.246890962	0	0	0	
450.672619	9.825091575	6.112957623	2.381866733	459.1588124	21061.4123	8.494824357	0	0	0	
451.2227955	3.516885553	6.114174969	1.50782272	162.3872159	20834.5743	7.621997688	0	0	0	
458.5372763	4.668489066	6.130220062	1.734922603	209.4065849	20567.8376	7.865142666	0	0	0	
734.7217176	36.14863749	6.600851946	3.614927095	1257.951809	25567.8935	10.21577904	0	0	0	
405.6852699	99.23924977	6.008039594	4.607559826	4639.899258	18967.6845	10.61559942	0	1	0	
752.696477	7.906052394	6.624989737	2.18673109	263.0470881	25043.4233	8.811720826	0	0	0	
448.8030303	7.23538961	6.108809777	2.108440674	311.0376094	19293.311	8.217250451	0	0	0	
468.3984697	4.430841672	6.151452023	1.692094126	182.6345425	19306.8827	7.843546149	0	0	0	
479.6134301	2968.038717	6.175063267	7.995993515	120671.855	19499.6925	14.17105678	0	1	0	
775.93083	45.41403162	6.655351325	3.837601819	1444.708787	24683.8708	10.49295314	0	0	0	
3008.873077	9.223076923	8.00965319	2.324647608	148.7176468	48516.6206	10.3343008	0	0	0	
380.2122549	7.42745098	5.943356319	2.131494351	335.4709186	17172.8033	8.074850671	0	0	0	
345.8905473	368.4328358	5.849009304	5.911968953	17324.18051	16264.2134	11.76097826	0	1	0	
5630.755556	16.91851852	8.636176494	2.885834732	196.5754214	65423.4675	11.52201123	0	0	0	
553.7612275	3.057634731	6.318537801	1.400600225	111.7606276	20240.7115	7.719138026	0	0	0	
368.3985873	0.278506559	5.911876243	0.245692646	12.39901692	16400.9793	6.157568889	0	0	0	
324.7191217	10.28087832	5.78603542	2.423109108	480.6791646	15182.1383	8.209144528	0	0	0	

Figure 5.1.2 Engineered Features

The figure 5.1.2 illustrates the feature-engineered dataset where additional behavioural attributes have been derived from the original data. Features such as tweets per day, followers per day, activity spikes, and combined activity scores are generated to better capture user behaviour patterns. These engineered features enhance the model's ability to distinguish between bot and human accounts by providing more informative and meaningful inputs.

4. Similarity-Based Graph Construction

src	dst	weight
1038481676	2592104616	0.999990495
2592104616	1038481676	0.999990495
1038481676	1424238228	0.999990225
1424238228	1038481676	0.999990225
1038481676	2963287552	0.999988597
2963287552	1038481676	0.999988597
1038481676	428641140	0.999982005
428641140	1038481676	0.999982005

Table 5.1.4 Similarity Graph Representation

The table 5.1.4 illustrates the construction of a similarity-based user graph, where each row represents a connection (edge) between two users identified as *source (src)* and *destination (dst)* along with an associated similarity weight. These connections are generated using k-

Nearest Neighbors (k-NN) with cosine similarity on normalized feature vectors, ensuring that users with similar behavioural and profile characteristics are linked together. The weight value indicates the strength of similarity between users, helping to capture relational patterns within the network. This graph structure enables the system to identify clusters of similar accounts, which is particularly useful for detecting coordinated bot activities.

5. Graph Embedding using GraphSAGE

Epoch	Loss
0	0.6769
10	0.1843
20	0.1600
30	0.1486

Table 5.1.5 GraphSAGE Training Loss Across Epochs

The table 5.1.5 illustrates the training progress of the GraphSAGE model, showing the reduction in loss values across multiple epochs. The decreasing loss indicates that the model is effectively learning meaningful patterns from the graph data. The final message confirms that the embeddings have been successfully generated and saved, making them ready for integration into the classification stage.

0	1	2	3	4	5	6
-1.55014	-0.89409	3.267738	-2.18971	-4.84098	2.615745	3.906266
-1.7365	-0.07659	3.67259	-2.66722	-4.25771	2.548207	4.043861
-1.28833	-1.00674	2.268898	-1.86357	-3.79799	1.881101	3.343835
-4.47123	1.616032	12.2304	-3.51791	-15.0814	4.62553	10.38595
-1.74594	1.140177	4.043302	-2.44986	-5.20815	1.511527	5.082751
-1.47677	-0.8305	3.18597	-1.50652	-3.63658	2.238774	3.362982
-0.38838	0.635714	0.313477	-1.07393	-0.22661	0.764344	1.530398

Table 5.1.6 Node Embeddings

The Table 5.1.6 illustrates the GraphSAGE-generated node embeddings, where each row corresponds to a user and each column represents a learned embedding dimension. These values are numerical representations capturing both the user's individual features and the influence of its neighboring nodes in the graph. The embeddings transform complex graph

relationships into a structured format that can be directly used for machine learning models, enabling better representation of behavioural and network patterns.

6. Model Training

```

MI selected 28 ML features
Final ML features after tree selection: 22

===== MODEL: XGBoost =====
Training setup: ML only
Training setup: Embeddings only
Training setup: ML + Embeddings

===== MODEL: RandomForest =====
Training setup: ML only
New best RandomForest model found. Saving...
Training setup: Embeddings only
New best RandomForest model found. Saving...
Training setup: ML + Embeddings
New best RandomForest model found. Saving...

Final Results
  Model          Setup Accuracy Precision Recall      F1      AUC
  XGBoost      ML only   0.93950   0.932735 0.861640 0.895780 0.971835
  XGBoost Embeddings only 0.94075   0.941712 0.856669 0.897180 0.972010
  XGBoost ML + Embeddings 0.94425   0.944846 0.865783 0.903588 0.975065
  RandomForest ML only   0.94225   0.958647 0.845070 0.898283 0.972589
  RandomForest Embeddings only 0.94200   0.950971 0.851698 0.898601 0.973139
  RandomForest ML + Embeddings 0.94350   0.954588 0.853355 0.901137 0.975548

Best RandomForest Setup: ML + Embeddings
Best RandomForest F1: 0.9011373578302713

```

Figure 5.1.3 Model Result

The figure 5.1.3 illustrates the performance evaluation and training output of the classification models. It shows the feature selection results, different training setups (ML only, embeddings only, and combined features), and the comparison between XGBoost and Random Forest models. The table highlights key evaluation metrics such as accuracy, precision, recall, F1-score, and AUC, along with the identification of the best-performing model configuration. This output demonstrates how combining machine learning features with graph embeddings improves overall classification performance.

5.2 WORKFLOW DIAGRAM

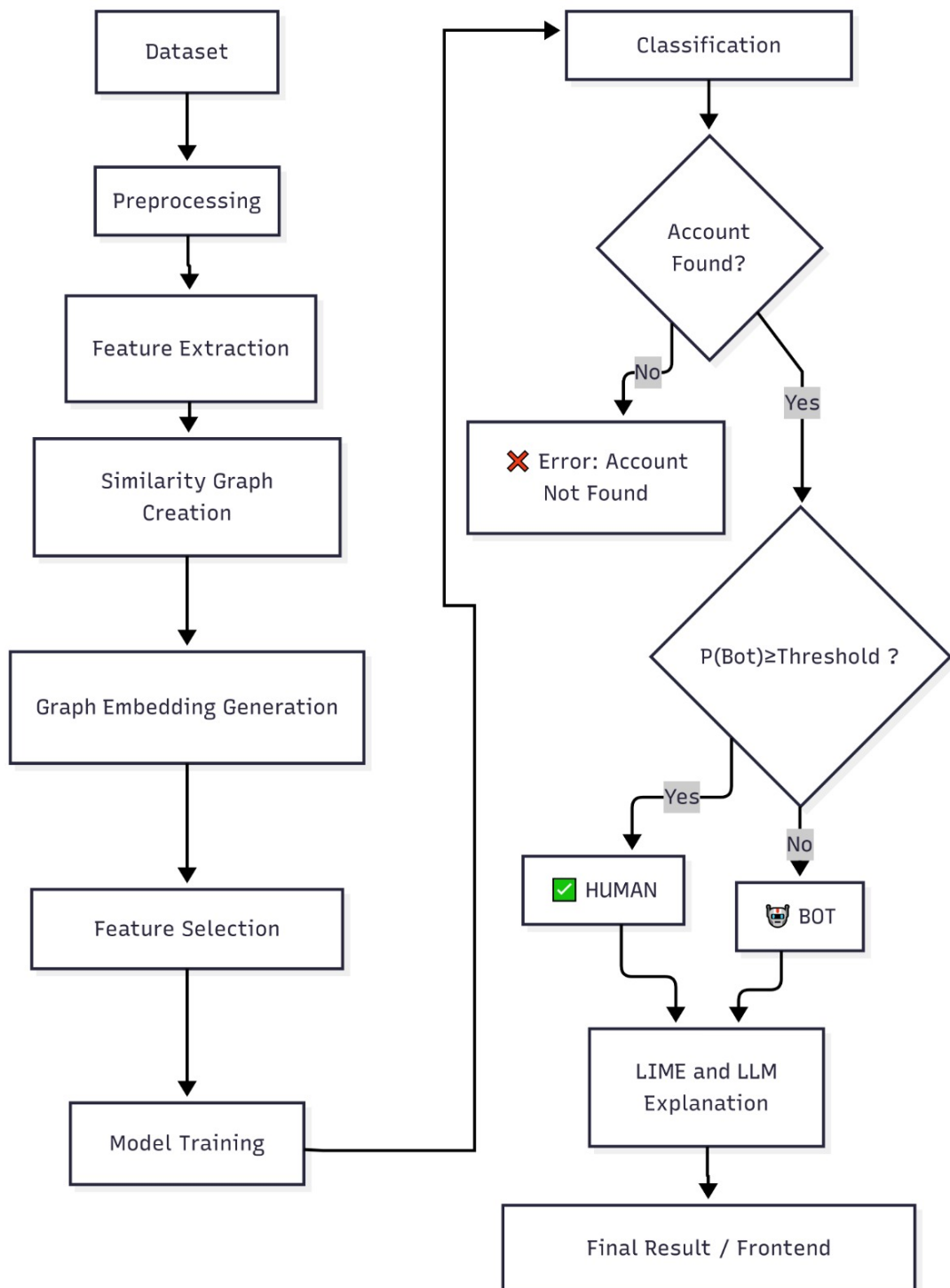


Figure 5.2.1 Workflow Diagram

The figure 5.2.1 shows the workflow which begins with the dataset, which contains the raw input data such as user activity and profile information. This data is first preprocessed to remove noise, handle missing values, and normalize formats so it becomes suitable for analysis. Feature extraction is then performed to convert the cleaned data into meaningful numerical attributes that represent user behavior. To capture relationships between users, a similarity graph is constructed where each node represents an account and edges indicate similarities or interactions. From this graph, embedding techniques are applied to generate vector representations that encode both individual characteristics and network structure, providing a richer feature set than traditional methods.

These extracted features and graph embeddings are then passed into the modeling phase, where feature selection identifies the most relevant attributes for prediction. A machine learning model is trained on this refined data to learn patterns and classify inputs, such as distinguishing between different types of accounts. Once trained, the model generates predictions for new data, which are further processed through an explainability component to interpret the reasoning behind each decision. Finally, the system outputs a result that combines both the prediction and its explanation, ensuring that the outcome is not only accurate but also transparent and understandable to users.

5.3 DATASET

The dataset used for training and evaluation was sourced from two publicly available benchmark collections, Cresci-17 and Twibot-22, both widely adopted in Twitter bot detection research. Cresci-17 provides labelled Twitter account records collected in 2017, containing a mix of genuine human accounts and several categories of bot accounts including social spambots and traditional spambots. Twibot-22 is a more recent and large-scale benchmark that reflects the different nature of bot behaviour on the platform, offering a diverse set of human and bot accounts with varied activity patterns. Each dataset provides structured user profile attributes and activity-based metrics alongside ground truth labels identifying each account as either bot or human.

Raw data was imported using pandas, with user identifiers preserved as strings to prevent truncation of long numeric IDs. Precomputed GraphSAGE node embeddings were loaded from a separate reference CSV file, with embedding column names standardised from the format `emb_0` to `emb_63` into integer indices for consistent referencing. User records, embeddings, and ground truth labels were merged on unique user identifiers, and integrity checks were performed to confirm correct alignment across all sources before passing the unified dataset to the preprocessing pipeline.

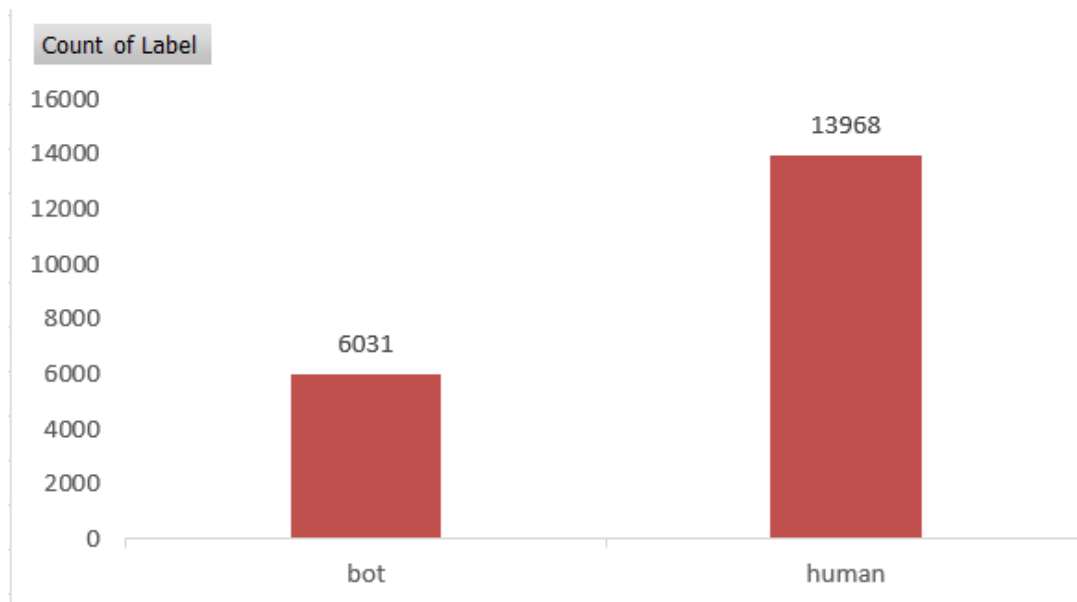


Figure 5.3.1 Class Distribution

The figure 5.3.1 illustrates the distribution of bot and human accounts in the dataset.

This chart shows the class distribution of the dataset, where human accounts (13,968) significantly outnumber bot accounts (6,031). This distribution is realistic and reflects real-world social media environments, where genuine users are naturally more prevalent than bots. However, this moderate imbalance may introduce a class imbalance problem, which can bias the model towards predicting the majority class (human). To address this, imbalance handling techniques such as class weighting (e.g., `scale_pos_weight` in XGBoost or `class_weight` in Random Forest) were applied during model training. This ensures that the model gives appropriate importance to the minority class (bots), improving its ability to accurately detect bot accounts despite the uneven distribution.

5.4 DATA PREPROCESSING

Raw account data was carefully cleaned to ensure accuracy, consistency, and reliability before being used for model training. Initially, numerical fields such as follower count, tweet count, and account age were validated to identify and correct invalid or unrealistic values. Negative values and extreme outliers were handled by clipping or replacing them with appropriate values, ensuring that the dataset reflects realistic user behaviour. In addition, special attention was given to fields like account age, where invalid ranges were removed to maintain logical consistency.

Missing entries were systematically handled by replacing them with suitable default or median values, ensuring that no null or undefined values were passed to the model. This step is crucial

because missing data can negatively affect model performance and lead to unreliable predictions. Duplicate records were also removed based on unique user identifiers, retaining the most relevant entries to maintain data integrity. Furthermore, categorical and boolean fields were standardized into numerical formats to ensure compatibility with machine learning algorithms.

A structured feature engineering pipeline was then applied to derive additional behavioural indicators directly from the cleaned data. These features capture deeper patterns such as user activity levels, engagement behaviour, and account characteristics, which cannot be fully represented using raw attributes alone. Logarithmic transformations and scaling techniques were also applied where necessary to handle skewed distributions and improve numerical stability. Overall, this preprocessing stage ensures that the dataset is clean, consistent, and enriched with meaningful features, enabling the model to learn more effectively and produce reliable results.

5.5 FEATURE ENGINEERING

The following behavioural features were engineered from the cleaned account data:

Tweets per Day = $\text{tweets_count} / \text{account_age_days}$

Followers per Day = $\text{followers_count} / \text{account_age_days}$

Follower-Following Ratio = $\text{followers_count} / (\text{friends_count} + 1)$

Follower Spike = $\text{followers_count} / \sqrt{\text{account_age_days}}$

Tweet Spike = $\text{tweets_count} / \sqrt{\text{account_age_days}}$

Extreme Activity Score = $\log(1 + \text{tweets_per_day}) + \log(1 + \text{followers_per_day})$

Young Account Flag = 1 if $\text{account_age_days} < 90$, else 0

Huge Followers Flag = 1 if $\text{followers_count} > 95\text{th percentile of training data}$, else 0

Young and Popular = $\text{Young Account Flag} * \text{Huge Follower Flag}$

Log Tweets per Day = $\log(1 + \text{tweets_per_day})$

Log Followers per Day = $\log(1 + \text{followers_per_day})$

- **Tweets per Day** – Represents how frequently a user posts content on average, helping identify highly active accounts.
- **Followers per Day** – Measures how quickly a user gains followers over time, indicating growth behaviour.
- **Follower-Following Ratio** – Shows the relationship between followers and following, useful for detecting imbalance in connections.
- **Follower Spike** – Highlights sudden or abnormal increases in follower count relative to account age.
- **Tweet Spike** – Captures unusually high posting activity compared to the account's lifetime.
- **Extreme Activity Score** – Combines posting and follower growth into a single metric to detect abnormal activity levels.
- **Young Account Flag** – Identifies recently created accounts that may be more likely to exhibit bot-like behaviour.
- **Huge Followers Flag** – Detects accounts with exceptionally high follower counts compared to typical users.
- **Young and Popular** – Flags accounts that are both new and rapidly gaining popularity, which can indicate suspicious growth.
- **Log Tweets per Day** – Applies logarithmic scaling to reduce the impact of extreme posting values.
- **Log Followers per Day** – Applies logarithmic scaling to normalize follower growth and reduce skewness.

5.6 NORMALISATION AND SCALING

Following feature engineering, normalisation was applied to ensure all numerical attributes were on a consistent scale before being passed to the classifier. Robust scaling was applied to the structured tabular features using a fitted RobustScaler, which scales each feature based on the interquartile range rather than mean and variance, making it resistant to the influence of outliers present in social media data. The RobustScaler transformation is defined as:

$$X_{scaled} = \frac{(X - Q2)}{(Q3 - Q1)}$$

where Q1, Q2, and Q3 denote the 25th, 50th, and 75th percentiles of the feature distribution respectively. Account age was scaled separately using a dedicated scaler to preserve its distinct distribution, as it exhibits significantly different statistical properties compared to the remaining tabular features.

GraphSAGE embedding vectors were L2-normalised to ensure consistent magnitude across all embedding dimensions, preventing any single dimension from disproportionately influencing the similarity computation or the downstream classifier. The L2 normalisation is defined as:

$$e_{norm} = \frac{e}{\sqrt{(e_1^2 + e_2^2 + \dots + e_{64}^2)}}$$

where e is the raw embedding vector and e_{norm} is the normalised output. All fitted scalers were serialised using joblib to ensure identical transformations are applied consistently at both training and inference time without any data leakage.

5.7 GRAPH EMBEDDING

Rather than training a graph neural network at inference time, the system performs a k-nearest neighbour lookup using cosine similarity over the training tabular reference to incorporate graph-based relational context. Cosine similarity between the input account feature vector and each training account is computed as:

$$cosine_similarity(A, B) = \frac{(A \cdot B)}{(||A|| \times ||B||)}$$

The ten most behaviourally similar training accounts are identified, and their corresponding precomputed GraphSAGE embeddings are retrieved and averaged to produce a single representative embedding vector:

$$e_{target} = \frac{1}{k} \sum_{i=1}^k e_i, \text{ where } k = 10$$

This averaged embedding is then L2-normalised and concatenated with the account's scaled tabular features to form the final combined feature vector:

$$X_{final} = concat(X_{scaled}, e_{target})$$

This approach allows the system to leverage social connectivity and structural network patterns alongside individual account features without requiring real-time graph construction or retraining of the graph neural network.

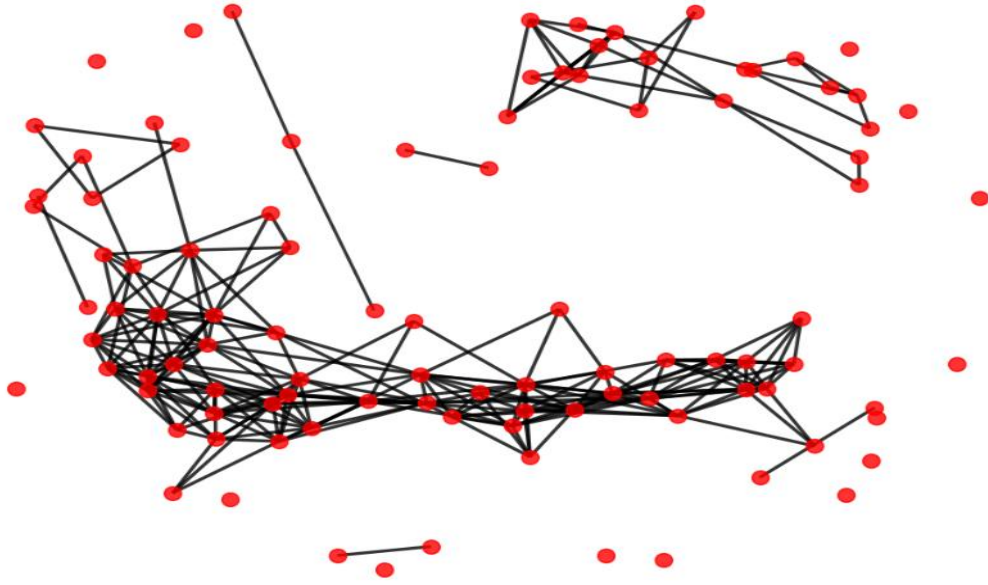


Figure 5.7.1 Bot Graph Representation

The figure 5.7.1 illustrates the relationships between bot accounts based on similarity, highlighting clusters and connectivity patterns.

This graph visualizes the similarity-based network of bot accounts, revealing important structural patterns. Multiple distinct clusters can be observed, indicating that bots do not follow a single behaviour but exist in different groups such as spam bots, follower bots, and hybrid types. A dense core cluster shows highly interconnected nodes, suggesting coordinated bot activity with very similar behavioural patterns. In addition, smaller clusters represent diverse bot strategies, while several isolated nodes indicate stealth or advanced bots that mimic human behaviour to avoid detection. Bridge connections between clusters suggest transitional or hybrid behaviours. Overall, this graph demonstrates that bot activity is not uniform and highlights the importance of graph-based methods like GraphSAGE in capturing these complex relationships for improved detection.

5.8 MODEL TRAINING

Feature selection was performed using mutual information scoring from scikit-learn, ranking all tabular attributes by their statistical relevance to the bot classification target. Mutual information between a feature X and the classification target Y is defined as:

$$I(X; Y) = \sum_{x \in X} \sum_{y \in Y} p(x, y) \log \left(\frac{p(x, y)}{p(x)p(y)} \right)$$

Features with higher mutual information scores were retained while redundant or low-information attributes were discarded. All sixty-four GraphSAGE embedding dimensions were appended to the selected tabular features to form the final combined feature vector.

Stage	Method Used	No. of Features
Initial	Total features	40
Stage 1	Mutual Information	28
Stage 2	Tree-Based (XGBoost)	22

Table 5.8.1 Feature Selection Process

The table 5.8.1 shows the features selected in each stage.

Feature selection was performed using a two-step approach to improve model efficiency and reduce noise. Initially, Mutual Information was used to evaluate the relevance of each feature with respect to the target variable, selecting the top 70% most informative features. Subsequently, a tree-based model (XGBoost) was applied to rank feature importance and further eliminate less significant attributes. This process reduced the feature space from 40 to 22 features, ensuring that only the most relevant behavioural and profile-based indicators were retained, thereby improving model performance and generalization.

The dataset was split into training and testing subsets using stratified sampling to maintain class proportions. A Random Forest classifier was trained on the combined feature set, where the final prediction is determined by aggregating votes across all decision trees:

$$y_{\text{predicted}} = \text{mode}\{h_1(X), h_2(X), \dots, h_T(X)\} \text{ , where } T = \text{total number of trees}$$

Class-weight balancing was applied to address the significant imbalance between bot and human accounts, assigning higher penalty to misclassification of the minority class. The optimal classification threshold was derived from precision-recall analysis on the training probability distribution by maximising the F1-score:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

theta = threshold value that maximises F1 on the training distribution

Model performance was evaluated across six configurations comparing XGBoost and Random Forest classifiers under three feature setups: tabular features only, GraphSAGE embeddings only, and the full combined representation. The Random Forest classifier with the combined setup achieved an accuracy of 94.53%, an F1-score of 90.36%, and an AUC of 97.37%, confirming that integrating graph-based embeddings with structured behavioural features meaningfully improves detection reliability over either component used in isolation.

5.9 LIME EXPLAINABILITY

Following classification, LIME was applied to each account to generate a local, per-account explanation of the classification decision. Unlike global explanation methods that describe the overall behaviour of a model, LIME produces explanations that are specific to each individual input, making it particularly suitable for account-level bot detection where different accounts may be flagged for entirely different reasons. LIME operates by perturbing the input features around the classified account, generating a set of neighbouring samples in the feature space, and observing how the model's predictions change in response to these perturbations. A lightweight interpretable model g is then fitted to these perturbed samples to approximate the Random Forest's local behaviour in the region surrounding the input account. The LIME objective is defined as:

$$\text{find } g \text{ that minimises: } \text{Loss}(f, g, \text{neighbourhood of } x) + \text{Complexity}(g)$$

where f is the Random Forest classifier, g is the interpretable explanation model, and the neighbourhood of x defines the local region around the input account. The loss term ensures that g closely approximates f within the local neighbourhood, while the complexity term penalises overly complicated explanations to keep them interpretable. In this system, LIME was configured to generate explanations by perturbing the input across 300 samples and identifying the fifteen most influential features, providing a comprehensive view of the signals driving each classification.

The resulting ranked feature list identifies each feature's directional contribution toward the classification:

$$\text{if } \text{weight}(f_i) > 0 \rightarrow \text{HUMAN signal}$$

$$\text{if } \text{weight}(f_i) < 0 \rightarrow \text{BOT signal}$$

where $\text{weight}(f_i)$ denotes the influence score assigned to feature f_i by the LIME explanation model. A higher absolute weight indicates a stronger influence on the classification outcome,

regardless of direction. The top five bot signals and top five human signals were extracted from the ranked list, sorted by their absolute influence scores, and translated into plain English descriptions using a feature description mapping that converts raw feature names and their conditions into human-readable phrases. This translation step ensures that the LIME output is meaningful to non-technical stakeholders who may not be familiar with the underlying feature engineering process, and prepares the evidence for submission to the language model in the subsequent stage.

5.10 LLM-BASED NARRATIVE GENERATION

The translated LIME signals were formatted into a structured prompt specifying the classification verdict, the confidence level derived from the probability score, and the top supporting bot and human evidence extracted from the LIME output. This prompt was designed to provide the language model with sufficient context to generate a coherent, balanced, and accurate narrative without referencing raw feature names, numerical scores, or model-specific terminology that would be inaccessible to a general audience. The prompt explicitly instructed the model to open with a clear verdict sentence, explain the behavioural patterns in plain English, acknowledge any contradictory signals naturally, and close with a calibrated confidence statement.

The prompt was submitted to a large language model via the OpenRouter API, which provides a unified interface for accessing hosted large language models without the need to manage local deployment or infrastructure. This approach enables seamless integration with cloud-based models while maintaining a simple API workflow similar to traditional HTTP-based services. In this system, the model used for narrative generation is `openai/gpt-oss-20b:free`, accessed through the OpenRouter platform. This model is designed to provide strong instruction-following capabilities and generate clear, structured explanations suitable for interpretability tasks. The Flask backend sends prompts to the OpenRouter API and retrieves generated responses over an HTTP connection, enabling efficient and scalable explanation generation while keeping the integration straightforward and flexible.

The LLM processed the structured prompt and generated a polished four to five sentence plain-language narrative summarising the classification reasoning in terms accessible to a non-technical reader. The narrative acknowledged any contradictory signals present in the LIME output, providing a balanced account of the evidence rather than a one-sided verdict, and included a calibrated confidence statement reflecting the margin between the bot and human probability scores. Open router hosted models ensured that no account data, LIME evidence, or classification output was transmitted to external services at any point during the

explanation generation process, preserving data privacy and enabling the complete system to operate in fully offline or air-gapped environments.

5.11 WEB APPLICATION DEPLOYMENT

The complete pipeline was exposed through a Flask backend API serving a browser-based frontend interface, enabling on-demand account analysis through a single unified web application. Flask was selected as the backend framework due to its lightweight architecture, straightforward routing, and seamless integration with the Python-based machine learning pipeline. All model artefacts including the trained Random Forest classifier, feature list, RobustScaler, age scaler, embedding normaliser, and the LIME explainer are loaded once at application startup rather than on each request, significantly reducing inference latency and ensuring consistent model state across all API calls. Cross-origin resource sharing was enabled via Flask-CORS to allow the browser-based frontend to communicate with the backend over HTTP without restriction, supporting deployment across different host and port configurations.

The backend exposes two primary API endpoints. The first endpoint returns the list of available user accounts from the dataset, enabling the frontend to validate inputs. The second endpoint receives a user ID, triggers the full sequential pipeline covering data retrieval, preprocessing, feature engineering, normalisation, k-nearest neighbour embedding lookup, Random Forest classification, LIME explanation generation, and LLM narrative generation, and packages all outputs into a structured JSON response returned to the frontend. Each stage of the pipeline is executed in a fixed order to ensure data consistency, and all intermediate outputs including raw features, scaled features, embedding vectors, probability scores, LIME rankings, and the LLM narrative are included in the final response object.

The browser-based frontend was built using HTML, CSS, and vanilla JavaScript without any external frontend framework dependencies, keeping the interface lightweight and easy to deploy alongside the Flask backend. Upon receiving the JSON response, the frontend renders the results across four dedicated tabs. The Account Data tab displays all thirty raw profile features extracted directly from the dataset, presented in a responsive grid layout with boolean fields rendered as colour-coded Yes or No indicators. The Classification tab presents the $P(\text{Human})$ and $P(\text{Bot})$ probability scores alongside the final verdict, displayed through animated probability bars with a threshold marker indicating the decision boundary. The LIME Evidence tab presents the ranked feature influence table with directional indicators distinguishing human and bot signals, proportional weight bars scaled relative to the maximum absolute influence score, and filter controls allowing users to isolate signals by direction. The

AI Summary tab displays the LLM-generated plain-language narrative in a formatted card layout, including metadata indicating the model used and the data source.

A collapsible sidebar panel is accessible through a fixed trigger button on the left edge of the screen, sliding into view with an animated transition and overlaying the main content without disrupting the page layout. The sidebar is divided into two collapsible sections the System Architecture section displays the seven-step pipeline with real-time status indicators that animate through each stage during analysis, and the Model Info section presents the key specifications of the deployed Random Forest model including performance metrics and classification threshold logic. This design ensures that the application remains uncluttered during normal use while providing full transparency into the system's operation for users who require it, making the interface accessible and informative to both technical researchers and non-technical platform administrators.

5.12 RESULT ANALYSIS

The performance of the bot detection framework was evaluated across six configurations, comparing XGBoost and Random Forest classifiers under three feature setups: tabular features only (ML only), graph embeddings only (Embeddings only), and the combined representation (ML + Embeddings). Results are summarised in the table below.

Model	Setup	Accuracy	Precision	Recall	F1	AUC
XGBoost	ML only	0.9395	0.9327	0.8616	0.8957	0.9718
XGBoost	Embeddings only	0.9407	0.9417	0.8566	0.8971	0.9720
XGBoost	ML + Embeddings	0.9442	0.9448	0.8657	0.9035	0.9750
RandomForest	ML only	0.9422	0.9586	0.8450	0.8982	0.9725
RandomForest	Embeddings only	0.9420	0.9509	0.8516	0.8986	0.9731
RandomForest	ML + Embeddings	0.9435	0.9545	0.8533	0.9011	0.9755

Table 5.12.1 Model Evaluation Metrics and Comparison

The table 5.12.1 presents a comparative evaluation of two models, XGBoost and Random Forest, across three different feature setups: machine learning (ML) features only, graph embeddings only, and a combination of both. Overall, both models achieve high performance across all metrics, with accuracy values consistently above 0.93 and AUC scores nearing 0.97 or higher. For XGBoost, the combined ML + embeddings setup yields the best performance, improving accuracy, F1-score, and AUC compared to using either feature type alone. A similar trend is observed for Random Forest, where the hybrid approach also delivers the strongest

overall results, indicating that combining traditional features with graph embeddings enhances predictive capability.

Across all configurations, the combined ML plus Embeddings setup consistently outperformed both the tabular-only and embeddings-only approaches, confirming that integrating GraphSAGE node embeddings with structured behavioural features produces a richer and more discriminative representation for bot detection.

Random Forest with the combined setup achieved the highest overall AUC of 0.9737, indicating strong separability between bot and human probability distributions across all thresholds. XGBoost with the combined setup achieved a marginally lower AUC of 0.9719 but demonstrated competitive F1 and adaptive accuracy scores. The embeddings-only configurations performed surprisingly well relative to the tabular-only baseline, demonstrating that graph-based relational context alone carries significant discriminative signal. The adaptive accuracy and adaptive F1 metrics, which reflect performance under the optimised classification threshold, remained consistent with standard metrics across all configurations, validating the stability of the derived threshold. Based on these results, Random Forest with the ML plus Embeddings configuration was selected as the final deployed model, offering the best balance of accuracy, F1-score, and AUC across the evaluation.

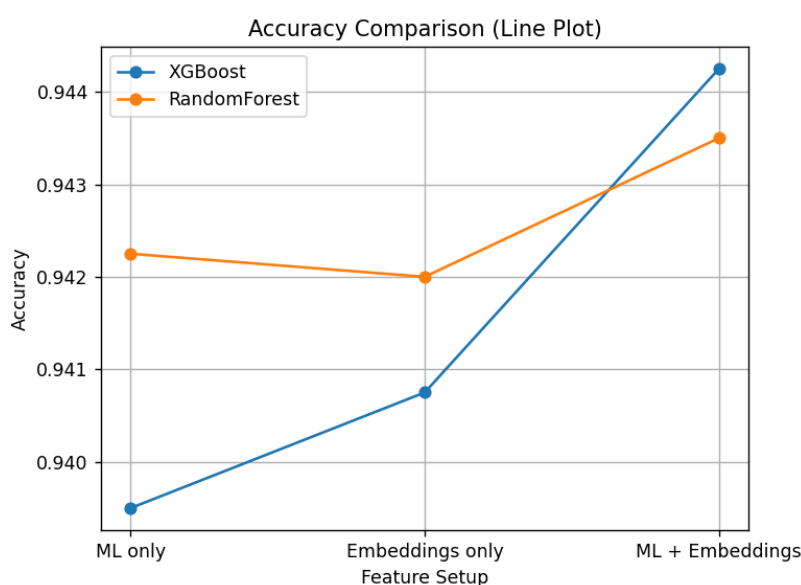


Figure 5.12.1 Feature Configuration Comparison

The figure 5.12.1 illustrate the comparison of accuracy between XGBoost and Random Forest across different feature setups.

This line plot compares the performance of XGBoost and Random Forest models across three feature configurations: ML only, embeddings only, and combined features. While XGBoost achieves slightly higher accuracy when both ML features and embeddings are used, the difference is minimal. Random Forest demonstrates more stable and consistent performance across all setups, making it more reliable for real-world deployment. It is less sensitive to hyperparameters, reduces the risk of overfitting, and handles mixed feature types effectively. Additionally, Random Forest is computationally simpler and easier to interpret, which is important for explainability in this project. Therefore, despite similar performance, Random Forest is preferred due to its robustness, simplicity, and better generalization capability.

5.12.1 OUTPUT ANALYSIS AND DISCUSSION

The system was evaluated to spot Twitter bots was put to the test. The goal was to see how well it could accurately identify bots, explain its reasoning, and hold up when faced with different types of data.

Feature	BOT	HUMAN
ff_ratio	3	3
followers_count	2	4
avg_polarity	2	1
avg_retweet	1	1
listed_count	2	3
tweet_count	1	1
unique_word_count	1	2
url_count	1	0
avg_mention	1	3
hashtag_count	1	0
has_profile_photo	0	1
profile_completeness	0	1
avg_subjectivity	0	1
mention_count	0	1
has_profile_url	0	1
avg_url	0	1
has_description	0	1

Table 5.12.1.1 Feature Analysis

The table 5.12.1.1 presents the key features that contribute to the classification of accounts into bot and human categories, highlighting how different behavioural and profile-based attributes influence the model's decision.

Bot pattern

Features that strongly indicate bot activity include `ff_ratio`, `followers_count`, and activity-related metrics such as number of tweets, hashtags, and URLs shared. Bots typically exhibit an imbalanced follower–following ratio, often following many accounts while having relatively fewer followers, or artificially inflated follower counts. In addition, they tend to show unusually high activity levels, such as frequent posting, excessive use of hashtags, and repeated sharing of URLs. These patterns reflect automated or scripted behaviour, which the model identifies as strong indicators of bot-like activity.

Human pattern

In contrast, features that contribute to identifying human users include `has_profile_photo`, `profile_completeness`, and `has_description`. Human accounts generally have well-maintained and complete profiles, including profile pictures, bios, and other descriptive information. Their activity patterns are more balanced and natural, reflecting genuine user behaviour rather than automated actions. These characteristics help the model distinguish authentic users from bots.

While certain features such as `followers_count` and `ff_ratio` appear in both bot and human classifications, their significance depends on how they interact with other features. It is not the presence of a single feature but the overall combination and pattern that determines the final classification. This contextual understanding enables the model to effectively differentiate between bots and humans, even when some individual attributes overlap.

FF Ratio Analysis

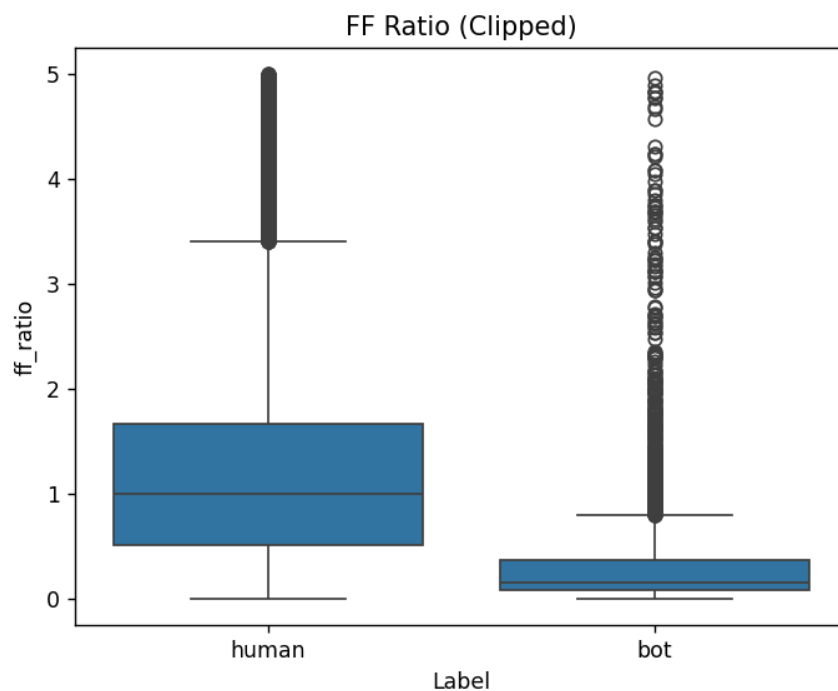


Figure 5.12.1.1 FF Ratio Comparison

The figure 5.12.1.1 illustrates the chart compares the follower-to-following ratio between bot and human accounts.

This boxplot shows the distribution of the follower-to-following ratio (ff_ratio) for bot and human accounts, highlighting clear behavioural differences between the two classes. Human accounts show a median value around 1.0 with a wider spread, indicating balanced and diverse interaction patterns. In contrast, bot accounts are heavily concentrated near lower values (around 0.15–0.2), suggesting that bots tend to follow many users but receive fewer followers. Although some overlap and outliers exist—where a few bots mimic human-like behaviour with higher ratios—the overall separation is evident. This confirms that ff_ratio is a strong discriminative feature, though not sufficient alone for classification. The clustering of bots near lower values and the variability among humans reflect real-world behaviour patterns, justifying its importance in feature engineering and its contribution to improving model performance.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENT

6.1 CONCLUSION

This project successfully designed and implemented an explainable, graph-based framework for detecting automated bot accounts on Twitter. By combining structured behavioural feature engineering with GraphSAGE-based node embeddings, the system captures both individual account characteristics and relational network context. A Random Forest classifier trained on this combined feature set achieved an AUC of 0.9737 and an accuracy of 94.53%, confirming that integrating graph embeddings with tabular features meaningfully improves classification reliability over either component in isolation.

A key contribution of the framework is its two-layer explainability pipeline, where LIME generates per-account feature-level explanations that are subsequently passed to a large language model to produce plain-language narrative summaries. This makes classification decisions transparent and accessible to both technical and non-technical stakeholders. The system was deployed as an interactive web application enabling on-demand analysis with real-time classification results, LIME evidence, and AI-generated explanations within a unified interface. Overall, this project demonstrates that combining supervised machine learning, graph-based embeddings, and explainable AI produces a detection framework that is accurate, transparent, and practically deployable.

6.2 FUTURE ENHANCEMENTS

1. Real-Time Data Integration using Twitter API

The current system relies on static datasets for analysis, which limits its ability to detect bots in real-world scenarios. As a future enhancement, integrating the Twitter API would enable real-time data collection and analysis of user accounts. This would allow the system to fetch live user activity, profile updates, and interaction patterns, making the detection process more dynamic and practical. Real-time integration would significantly improve the system's relevance and effectiveness in identifying newly emerging bot behaviours.

2. Multimedia and URL-Based Analysis

At present, the system focuses mainly on textual and behavioural features. Future improvements can include analyzing multimedia content such as images and videos, as well as evaluating URLs shared in tweets. By incorporating computer vision techniques and link

analysis, the system can detect suspicious patterns like repeated media usage, malicious links, or spam content. This multi-modal analysis would provide a deeper understanding of user activity and improve the overall accuracy of bot detection.

3. Extension to Multiple Social Media Platforms

While the current framework is designed for Twitter data, it can be extended to other platforms such as Instagram and Reddit. The modular architecture of the system makes it adaptable for integrating platform-specific features and data formats. Expanding the system across multiple platforms would allow broader detection of bot activity and coordinated campaigns, making the solution more versatile and applicable in diverse social media environments.

CHAPTER 7

REFERENCES

- [1] D. Javed, N. Z. Jhanjhi, N. A. Khan, S. K. Ray, A. Al-Dhaqm, and V. R. Kebande, "Identification of spambots and fake followers on social network via interpretable AI-based machine learning," *IEEE Access*, vol. 13, pp. 52246–52259, 2025, doi: 10.1109/ACCESS.2025.3551993.
- [2] A. Khan, A. Fatima, R. Jamil, H. Ahmed, and A. Saba, "Enhancing social media bot detection with cross-feature gating and residual learning," *ICCK Trans. Emerg. Top. Artif. Intell.*, vol. 3, no. 1, pp. 20–32, 2026, doi: 10.62762/TETAI.2025.791029.
- [3] L. H. X. Ng and K. M. Carley, "A global comparison of social media bot and human characteristics," *Sci. Rep.*, vol. 15, no. 10973, pp. 1–18, 2025, doi: 10.1038/s41598-025-96372-1.
- [4] A. Sallah, S. Agoujil, M. A. Wani, M. Hammad, Y. Maleh, and A. A. Abd El-Latif, "Fine-tuned understanding: Enhancing social bot detection with transformer-based classification," *IEEE Access*, vol. 12, pp. 118250–118269, 2024, doi: 10.1109/ACCESS.2024.1234567.
- [5] Z. Ellaky, F. Benabbou, Y. Matrane, and S. Qaqa, "A hybrid deep learning architecture for social media bots detection based on BiGRU-LSTM and GloVe word embedding," *IEEE Access*, vol. 12, pp. 100278–100292, 2024, doi: 10.1109/ACCESS.2024.3430859.
- [6] R. Kenny, B. Fischhoff, A. Davis, and C. Canfield, "Improving social bot detection through aid and training," *Human Factors*, vol. 66, no. 10, pp. 2323–2344, 2024, doi: 10.1177/00187208231210145.
- [7] K. Hayawi, S. Saha, M. M. Masud, S. S. Mathew, and M. Kaosar, "Social media bot detection with deep learning methods: A systematic review," *Neural Comput. Appl.*, vol. 35, pp. 8903–8918, 2023, doi: 10.1007/s00521-023-08352-z.
- [8] A. Aguilera, P. Quinteros, I. Dongo, and Y. Cardinale, "CrediBot: Applying bot detection for credibility analysis on Twitter," *IEEE Access*, vol. 11, pp. 108365–108380, 2023, doi: 10.1109/ACCESS.2023.3320687.

- [9] A. Dehghan et al., "Detecting bots in social-networks using node and structural embeddings," *J. Big Data*, vol. 10, no. 119, pp. 1–37, 2023, doi: 10.1186/s40537-023-00796-3.
- [10] I. Mbona and J. H. P. Eloff, "Classifying social media bots as malicious or benign using semi-supervised machine learning," *J. Cybersecur.*, vol. 9, no. 1, Art. no. tyac015, 2023, doi: 10.1093/cybsec/tyac015.
- [11] M. Aljabri, R. Zagrouba, A. Shaahid, F. Alnasser, A. Saleh, and D. M. Alomari, "Machine learning-based social media bot detection: A comprehensive literature review," *Soc. Netw. Anal. Min.*, vol. 13, no. 20, pp. 1–40, Jan. 2023, doi: 10.1007/s13278-022-01020-5.

CHAPTER 8

APPENDIX

8.1 SOURCE CODE

"""

Bot Detection API — Flask backend

Install: pip install flask flask-cors

Run: python app.py

Then open index.html in your browser.

"""

```
import os, time, requests
```

```
import pandas as pd
```

```
import numpy as np
```

```
import joblib
```

```
from flask import Flask, jsonify
```

```
from flask_cors import CORS
```

```
from sklearn.neighbors import NearestNeighbors
```

```
import lime, lime.lime_tabular
```

```
import random
```

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
app = Flask(__name__, static_folder='.', static_url_path=)
```

```
CORS(app)
```

```
# — CONFIG
```

```
OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")
```

```
LLM_MODEL = "openai/gpt-oss-20b:free"
```

```
LLM_MAX_TOKENS = 300
```

```
LLM_TEMPERATURE = 0.3
```

```
LLM_RETRY_BASE_DELAY = 10
```

```
LLM_RETRIES = 5
```

```
USE_LOCAL_MODEL = os.getenv("USE_LOCAL_MODEL", "false").strip().lower() == "true"
```

```
OLLAMA_MODEL = "qwen2:7b"
```

```
OLLAMA_BASE_URL = "http://localhost:11434"
```

```
print(f"LLM mode: {'LOCAL (Ollama/' + OLLAMA_MODEL + '}' if USE_LOCAL_MODEL else  
'API (OpenRouter/' + LLM_MODEL + ')}")
```

— CLASSIFICATION THRESHOLD

Optimal threshold derived from ROC/precision-recall analysis.

P(Human) >= 0.4237 → HUMAN

P(Human) < 0.4237 → BOT

THRESHOLD = 0.39

```
def classify(prob: float) -> str:
```

```
    return "HUMAN" if prob >= THRESHOLD else "BOT"
```

— LOAD ARTIFACTS

print("Loading model artifacts...")

```
model      = joblib.load("Models/random_forest_bot_classifier.pkl")
```

```
feature_list = joblib.load("Models/random_forest_feature_list.pkl")
```

```
ml_cols     = joblib.load("Models/random_forest_ml_feature_cols.pkl")
```

```
robust_scaler = joblib.load("Models/robust_scaler.pkl")
```

```
age_scaler   = joblib.load("Models/age_scaler.pkl")
```

```
emb_normalizer = joblib.load("Models/embedding_normalizer.pkl")
```

```
ref_emb      = pd.read_csv("Dataset/training_embeddings_reference.csv")
```

```
train_tabular = pd.read_csv("Dataset/training_tabular_reference.csv")
```

```
api_df = pd.read_excel("api_file.xlsx", dtype={"user_id": str})
```

```
rename_map = {"emb_{i}": str(i) for i in range(64)}
```

```
ref_emb = ref_emb.rename(columns=rename_map)
```

```
emb_cols = [c for c in ref_emb.columns if c.isdigit()]
```

```
DROP_SIM_COLS = ["user_id", "user_name", "Label", "dataset"]
```

```
similarity_cols = [c for c in train_tabular.columns if c not in DROP_SIM_COLS]
```

```
nn = NearestNeighbors(n_neighbors=10, metric="cosine")
```

```
nn.fit(train_tabular[similarity_cols])
```

```
followers_95_quantile = train_tabular["followers_count"].quantile(0.95)
```

```
RAW_INPUT_COLS = [c for c in api_df.columns if c not in
```

```
    ["user_id", "user_name", "Label", "label", "dataset"]]
```

```
binary_cols = ["verified", "has_description", "has_prof_url", "has_location",
```

```
    "has_prof_img", "young_account_flag", "huge_followers_flag",  
    "young_and_popular"]
```

```
# Engineered features (derived during preprocessing)
```

```
ENGINEERED_FEATURES = [
```

```
    "tweets_per_day", "followers_per_day", "log_tweets_per_day", "log_followers_per_day",
```

```

    "followers_spike", "tweet_spike", "extreme_activity_score",

    "young_account_flag", "huge_followers_flag", "young_and_popular"

]

user_id_to_idx = {uid: i for i, uid in enumerate(ref_emb["user_id"].values)}

train_user_ids = train_tabular["user_id"].values

# Map each training row → its embedding row index (-1 if missing)

train_to_emb_idx = np.array([

    user_id_to_idx.get(uid, -1) for uid in train_user_ids

])

emb_matrix = ref_emb[emb_cols].values.astype(np.float32)

# Cache: user_id -> unnormalized 64-d embedding vector (before emb_normalizer)

embedding_cache = {}

# — LIME SETUP


---




---



def _build_background():

    bg = pd.DataFrame(index=train_tabular.index)

    for c in RAW_INPUT_COLS:

```

```

        bg[c] = train_tabular[c] if c in train_tabular.columns else 0

    return bg.replace([np.inf, -np.inf], 0).fillna(0).values

background_data = _build_background()

bg_medians    = pd.DataFrame(background_data, columns=RAW_INPUT_COLS).median()

cat_indices    = [i for i, n in enumerate(RAW_INPUT_COLS) if n in binary_cols]

explainer = lime.lime_tabular.LimeTabularExplainer(

    training_data=background_data, feature_names=RAW_INPUT_COLS,

    class_names=["BOT", "HUMAN"], categorical_features=cat_indices,

    mode="classification", discretize_continuous=True, random_state=42,

)

print("Ready — http://localhost:5000")

def _add_engineered_features(df: pd.DataFrame) -> pd.DataFrame:

    """Add engineered features in place on a dataframe that already has clipped raw
    counts."""

    df["tweets_per_day"]    = df["tweets_count"] / df["account_age_days"]

    df["followers_per_day"]  = df["followers_count"] / df["account_age_days"]

    df["log_tweets_per_day"] = np.log1p(df["tweets_per_day"])

    df["log_followers_per_day"] = np.log1p(df["followers_per_day"])

    df["followers_spike"]    = df["followers_count"] / np.sqrt(df["account_age_days"])

```

```

df["tweet_spike"]      = df["tweets_count"] / np.sqrt(df["account_age_days"])

df["extreme_activity_score"] = df["log_followers_per_day"] + df["log_tweets_per_day"]

df["young_account_flag"]  = (df["account_age_days"] < 90).astype(int)

df["huge_followers_flag"] = (df["followers_count"] > followers_95_quantile).astype(int)

df["young_and_popular"]   = df["young_account_flag"] * df["huge_followers_flag"]

return df

```

```

def _compute_embedding_means(df: pd.DataFrame) -> np.ndarray:

```

```

    """Compute mean embedding vectors for each row using k-NN similarity lookup."""

```

```

    for c in similarity_cols:

```

```

        if c not in df.columns:

```

```

            df[c] = 0

```

```

    sim_data = df[similarity_cols].replace([np.inf, -np.inf], 0).fillna(0)

```

```

    _, idx_matrix = nn.kneighbors(sim_data)

```

```

    emb_idx_matrix = train_to_emb_idx[idx_matrix]

```

```

    valid_mask = emb_idx_matrix >= 0

```

```

    safe_idx = np.where(valid_mask, emb_idx_matrix, 0)

```

```

    fetched = emb_matrix[safe_idx]

```

```
    fetched = np.where(valid_mask[:, :, None], fetched, 0.0)
```

```
    count = valid_mask.sum(axis=1, keepdims=True).clip(min=1)
```

```
    return fetched.sum(axis=1) / count
```

```
def get_cached_user_embedding(user_id: str, row: pd.DataFrame) -> np.ndarray:
```

```
    """Return cached unnormalized embedding for a user, computing it once if needed."""
```

```
    if user_id in embedding_cache:
```

```
        return embedding_cache[user_id]
```

```
    df = row.copy()
```

```
    df["account_age_days"] = df["account_age_days"].clip(lower=1.0)
```

```
    df["tweets_count"] = df["tweets_count"].clip(lower=0.0)
```

```
    df["followers_count"] = df["followers_count"].clip(lower=0.0)
```

```
    _add_engineered_features(df)
```

```
    emb_means = _compute_embedding_means(df)
```

```
    embedding_cache[user_id] = emb_means[0].astype(np.float32)
```

```
    return embedding_cache[user_id]
```

```
def preprocess(df: pd.DataFrame, fill_nulls: bool = False, fixed_embedding: np.ndarray = None) -> pd.DataFrame:
```

```
df = df.copy()
```

```
if fill_nulls:
```

```
    for col in binary_cols:
```

```
        if col in df.columns:
```

```
            df[col] = df[col].fillna(0)
```

```
df = df.fillna(bg_medians)
```

```
# — 1. Clip raw counts
```

```
df["account_age_days"] = df["account_age_days"].clip(lower=1.0)
```

```
df["tweets_count"] = df["tweets_count"].clip(lower=0.0)
```

```
df["followers_count"] = df["followers_count"].clip(lower=0.0)
```

```
# — 2. Engineered features
```

```
_add_engineered_features(df)
```

```
# — 3. KNN neighbour lookup + embedding (fully vectorised)
```

```
if fixed_embedding is not None:
```

```
    emb_vec = np.asarray(fixed_embedding, dtype=np.float32).reshape(1, -1)
```

```
    emb_means = np.repeat(emb_vec, len(df), axis=0)
```

else:

```
emb_means = _compute_embedding_means(df)
```

```
emb_df = pd.DataFrame(emb_means, columns=emb_cols, index=df.index)
```

```
df = pd.concat([df, emb_df], axis=1)
```

— 4. Scale embeddings

```
df[emb_cols] = df[emb_cols].replace([np.inf, -np.inf], np.nan).fillna(0)
```

```
df[emb_cols] = emb_normalizer.transform(df[emb_cols])
```

— 5. Robust scaler

```
robust_cols = robust_scaler.feature_names_in_
```

```
for c in robust_cols:
```

```
    if c not in df.columns:
```

```
        df[c] = 0
```

```
df[robust_cols] = robust_scaler.transform(df[robust_cols])
```

— 6. Age scaler

```
df[["account_age_days"]] = age_scaler.transform(df[["account_age_days"]])
```

— 7. Ensure all model features exist

```
for c in feature_list:

    if c not in df.columns:

        df[c] = 0
```

```
return df
```

```
def get_engineered_features_unscaled(row: pd.DataFrame) -> dict:
```

```
    """Extract engineered features WITHOUT applying scaling transformations."""
```

```
    df = row.copy()
```

```
    # Clip raw counts
```

```
    df["account_age_days"] = df["account_age_days"].clip(lower=1.0)
```

```
    df["tweets_count"] = df["tweets_count"].clip(lower=0.0)
```

```
    df["followers_count"] = df["followers_count"].clip(lower=0.0)
```

```
    # Calculate engineered features
```

```
    _add_engineered_features(df)
```

```
    # Extract values as native Python types
```

```
    engineered_values = {}
```

```

for feat in ENGINEERED_FEATURES:

    v = df[feat].values[0] if feat in df.columns else None

    if hasattr(v, 'item'):

        v = v.item()

    if isinstance(v, float) and (v != v or v == float('inf') or v == float('-inf')):

        v = None

    engineered_values[feat] = v


return engineered_values

```

— CORE FUNCTIONS

```

def run_pipeline(row: pd.DataFrame) -> float:

    processed = preprocess(row, fill_nulls=False)

    return model.predict_proba(processed[feature_list])[:, 1][0]


def get_lime_features(row, fixed_embedding: np.ndarray = None):

    lr = row[RAW_INPUT_COLS].copy()

    for col in binary_cols:

        if col in lr.columns: lr[col] = lr[col].fillna(0)

    lr = lr.fillna(bg_medians)

    random.seed(42)

```

```
np.random.seed(42)
```

```
def lime_predict(raw_array: np.ndarray) -> np.ndarray:
```

```
    df = pd.DataFrame(raw_array, columns=RAW_INPUT_COLS)
```

```
    processed = preprocess(df, fill_nulls=True, fixed_embedding=fixed_embedding)
```

```
    probs = model.predict_proba(processed[feature_list])[:, 1]
```

```
    return np.column_stack([1 - probs, probs])
```

```
# Request all features, not just top 15
```

```
exp = explainer.explain_instance(
```

```
    data_row=lr.values[0].astype(float), predict_fn=lime_predict,
```

```
    num_features=len(RAW_INPUT_COLS), num_samples=300, labels=(1,)
```

```
)
```

```
return [{"rank": i+1, "feature": f, "weight": round(w, 6),
```

```
        "direction": "HUMAN" if w > 0 else "BOT"}]
```

```
for i, (f, w) in enumerate(exp.as_list(label=1))]
```

```
FEATURE_DESC = {
```

```
    "followers_count": "number of followers", "friends_count": "number of accounts followed",
```

```
    "tweets_count": "total tweets posted", "listed_count": "times added to Twitter lists",
```

```
    "hashtag_count": "total hashtags used", "mentions_count": "total mentions of other users",
```

```

"retweet_count":"total retweets received","reply_count":"total replies received",

"url_count":"total URLs shared","ff_ratio":"follower-to-following ratio",

"avg_hashtag":"avg hashtags per tweet","avg_mentions":"avg mentions per tweet",

"avg_retweet":"avg retweets per tweet","avg_reply":"avg replies per tweet",

"avg_url":"avg URLs per tweet","avg_user_engagement":"avg user engagement score",

"has_description":"account has a bio","has_prof_url":"account has a profile URL",

"has_location":"account has a location set","has_prof_img":"account has a profile image",

"profile_completeness":"profile completeness","avg_polarity":"avg tweet sentiment
polarity",

"avg_subjectivity":"avg tweet subjectivity","unique_word_count":"total unique words used",

"unique_word_use":"ratio of unique words","punctuation_count":"total punctuation used",

"avg_sentence_length":"avg sentence length","punctuation_density":"punctuation
density",

"account_age_days":"account age in days","verified":"account is verified",

"tweets_per_day":"avg tweets per day","followers_per_day":"avg follower gain per day",

"followers_spike":"follower spike vs account age","tweet_spike":"tweet spike vs account
age",

"extreme_activity_score":"combined activity spike score",

"young_account_flag":"account under 90 days old",

"huge_followers_flag":"follower count in top 5%","young_and_popular":"new and high-
follower account",

}

```

```

def _h(raw):

    for key, desc in FEATURE_DESC.items():

        if raw.startswith(key) or f" {key} " in raw or raw.endswith(key):

            cond = raw.replace(key, "").strip().lstrip("=").strip()

            return f"{desc} ({cond})" if cond else desc

    return raw


def get_llm_explanation(user_id, prediction, p_human, lime_features):

    bots = [(f["feature"], f["weight"]) for f in lime_features if f["weight"] < 0]

    humans = [(f["feature"], f["weight"]) for f in lime_features if f["weight"] > 0]

    bot_lines = "\n".join(f" - {_h(f)} (influence: {abs(w):.4f})" for f, w in
sorted(bots, key=lambda x: x[1])[:5])

    human_lines = "\n".join(f" - {_h(f)} (influence: {abs(w):.4f})" for f, w in sorted(humans,
key=lambda x: -x[1])[:5])

    if prediction == "HUMAN":

        verdict = "likely human"

        confidence = "narrow" if p_human <= 0.46 else "moderate"

    else: # BOT

        verdict = "likely automated (bot)"

        confidence = "narrow" if p_human >= 0.38 else "moderate"

```

prompt = f"""You are an expert analyst writing a clear, structured report on a Twitter account's authenticity.

A machine learning model analysed account {user_id} and concluded it is {verdict}, with a {confidence} margin of confidence.

Evidence suggesting automated (bot) behaviour:

{bot_lines or " - None identified"}

Evidence suggesting genuine human behaviour:

{human_lines or " - None identified"}

Write a structured analysis as exactly 4 bullet points that a non-technical reader would understand. Format each bullet point starting with "• " on its own line.

Rules:

- Use **double asterisks** around key terms that deserve emphasis: the verdict (e.g. **likely a bot** or **likely human**), the confidence level (e.g. **moderate confidence** or **narrow margin**), and the 1-2 most important behavioural signals.
- Do NOT mention feature names, numbers, or model scores.
- Bullet 1: Clear verdict — state plainly whether this account appears to be **human**, a **bot**, or is **uncertain**, and at what confidence level.
- Bullet 2: Main behavioural patterns that led to this conclusion (plain English).

- Bullet 3: Any contradictory or mixed signals, if present. If none, note one additional supporting pattern.
- Bullet 4: A brief caveat about confidence and what this analysis can and cannot tell us.

Professional, journalistic tone. Complete every sentence. Each bullet must be 1-2 sentences. """

```
if USE_LOCAL_MODEL:
```

```
    return _llm_ollama(prompt)
```

```
else:
```

```
    return _llm_openrouter(prompt)
```

```
def _llm_ollama(prompt: str) -> str:
```

```
    """Call local Ollama instance."""
```

```
    url = f"{OLLAMA_BASE_URL}/api/generate"
```

```
    payload = {
```

```
        "model": OLLAMA_MODEL,
```

```
        "prompt": prompt,
```

```
        "stream": False,
```

```
        "options": {"temperature": LLM_TEMPERATURE, "num_predict": LLM_MAX_TOKENS}
```

```
    }
```

```
    try:
```

```
        r = requests.post(url, json=payload, timeout=120)
```

```
        if r.status_code == 200:
```

```

text = r.json().get("response", "").strip()

if text:

    return text

print(f"Empty Ollama response: {r.json()}", flush=True)

else:

    print(f'Ollama error {r.status_code}: {r.text}', flush=True)

except Exception as e:

    print(f'Ollama request failed: {e}', flush=True)

return "LLM explanation unavailable."


def _llm_openrouter(prompt: str) -> str:

    """Call OpenRouter API with retries."""

    headers = {"Authorization": f"Bearer {OPENROUTER_API_KEY}",

               "Content-Type": "application/json",

               "HTTP-Referer": "https://bot-detection"}

    payload = {"model": LLM_MODEL, "max_tokens": LLM_MAX_TOKENS, "temperature":

LLM_TEMPERATURE,

               "messages": [{"role": "user", "content": prompt}]}

    for attempt in range(1, LLM_RETRIES + 1):

        try:

            r = requests.post("https://openrouter.ai/api/v1/chat/completions",

```

```

        headers=headers, json=payload, timeout=60)

    if r.status_code == 200:

        choices = r.json().get("choices", [])

        if not choices:

            print(f"Empty choices: {r.json()}", flush=True)

        if choices:

            content = choices[0].get("message", {}).get("content")

            text = content.strip() if isinstance(content, str) else ""

            if text:

                return text

        if attempt < LLM_RETRIES:

            time.sleep(LLM_RETRY_BASE_DELAY)

            continue

        elif r.status_code == 429:

            time.sleep(LLM_RETRY_BASE_DELAY * (2 ** (attempt - 1)))

        else:

            return f"LLM error {r.status_code}: {r.text}"

    except Exception as e:

        return f"Request failed: {e}"

    return "LLM explanation unavailable."

```

— ROUTES

```
@app.route("/api/users")
```

```
def list_users():
```

```
    users = (api_df[["user_id", "user_name"]]  
              .dropna(subset=["user_id"])  
              .assign(user_id=lambda d: d["user_id"].astype(str))  
              .drop_duplicates("user_id"))  
  
    return jsonify(users.to_dict(orient="records"))
```

```
@app.route("/api/analyse/<user_id>")
```

```
def analyse(user_id):
```

```
    row = api_df[api_df["user_id"] == str(user_id)]  
  
    if row.empty:  
        return jsonify({"error": f"User {user_id} not found"}), 404  
  
    row = row.iloc[[0]].copy()  
  
    user_name = str(row["user_name"].values[0]) if "user_name" in row.columns else user_id  
  
    prob = run_pipeline(row)  
  
    label = classify(prob)  
  
    cached_embedding = get_cached_user_embedding(user_id, row)  
  
    lime_features = get_lime_features(row, fixed_embedding=cached_embedding)
```

```

llm_text    = get_llm_explanation(user_id, label, prob, lime_features)

# Raw input features for frontend display (exclude ID/label cols)

raw_display_cols = [c for c in RAW_INPUT_COLS if c not in

                    ["user_id", "user_name", "Label", "label", "dataset"]]

raw_features = {}

for c in raw_display_cols:

    v = row[c].values[0] if c in row.columns else None

    # convert numpy types to native Python for JSON serialisation

    if hasattr(v, 'item'):

        v = v.item()

    # replace NaN / Inf with None so Flask produces valid JSON

    if isinstance(v, float) and (v != v or v == float('inf') or v == float('-inf')):

        v = None

    raw_features[c] = v


# Extract engineered feature values BEFORE scaling

engineered_values = get_engineered_features_unscaled(row)

return jsonify({

    "user_id": user_id, "user_name": user_name,

    "prediction": label, "p_human": round(prob, 4), "p_bot": round(1-prob, 4),

```

```
"lime_features": lime_features, "llm_explanation": llm_text,  
  
"raw_features": raw_features, "engineered_features": engineered_values,  
  
})
```

```
from flask import send_from_directory
```

```
@app.route('/')
```

```
def index():
```

```
    return send_from_directory('.', 'index.html')
```

```
if __name__ == "__main__":
```

```
    app.run(debug=False, port=5000)
```

8.2 PROJECT SCREENSHOTS



Figure 8.2.1 Frontend and User input page

The figure 8.1.1 shows the Frontend page of the Twitter bot detection including an Input bar.

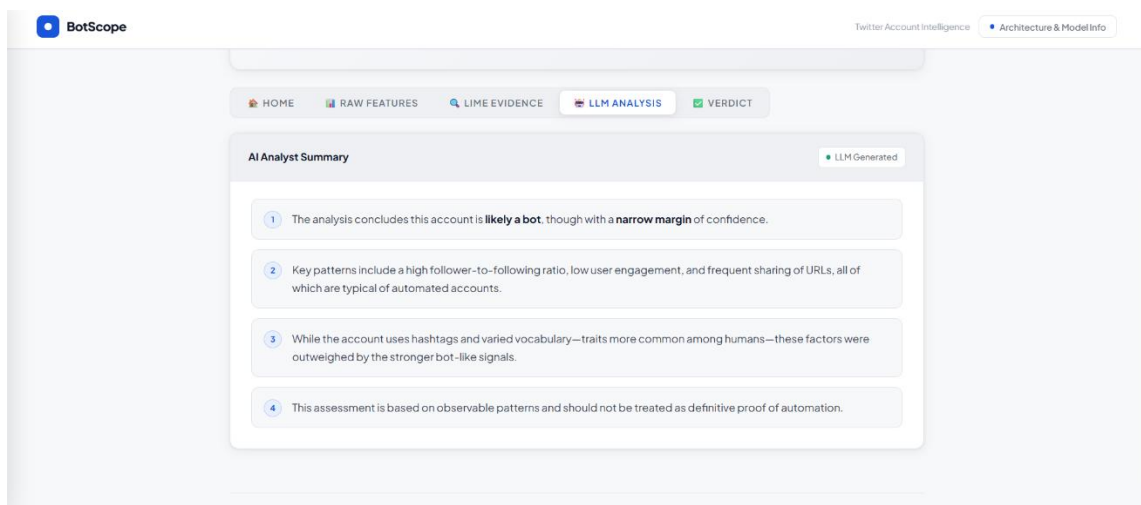


Figure 8.2.2 Frontend – LLM Summary

The figure 8.1.2 Shows the Large language Model summarization of the LIME results, Providing overall explainability

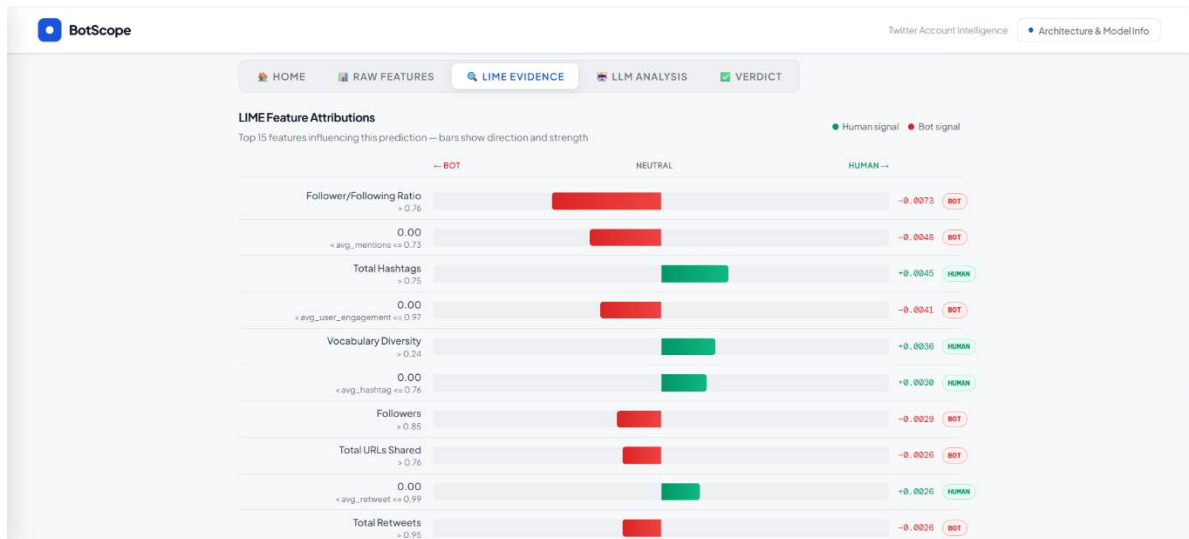


Figure 8.2.3 Frontend – LIME Results

The figure 8.1.3 Shows the LIME results, and explaining the ML Model's Features whether which feature is pushing an account towards human or Bot

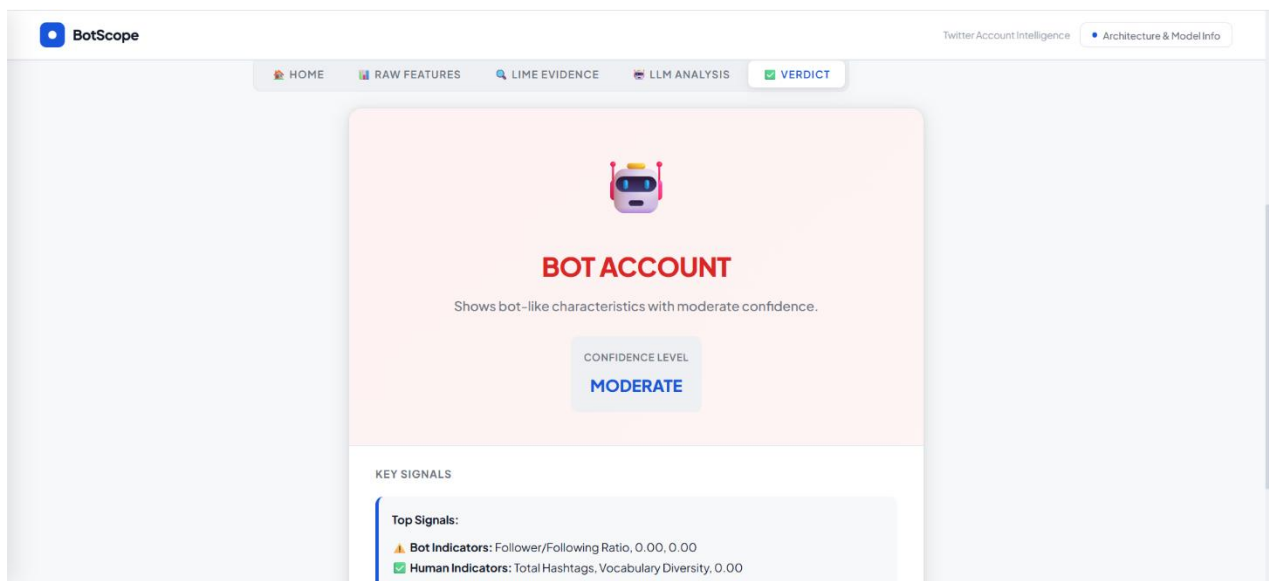


Figure 8.2.4 Final verdict

The figure 8.1.4 shows the final verdict of user as whether it is a bot or human with confidence level and key signals.

TECHNICAL BIOGRAPHY



Ajay M (220171601012), born in the 11th october 2004 in Chennai, Tamilnadu, India. He completed his 12th Grade in the year 2022. He is currently pursuing B. Tech in Artificial Intelligence and Data science at the School of Computer Science and Engineering at B.S. Abdur Rahman Crescent Institute of Science and Technology which is located at Vandalur, Chennai, India. His areas of interest lies in areas like Machine Learning, LLM-Ops, and Prompt Engineering. His email id is ajayym2025@gmail.com .



Abdul Rahim S (220171601003), born in the 3rd june 2005 in Chennai, Tamilnadu, India. He completed his 12th Grade in the year 2022. He is currently pursuing B. Tech in Artificial Intelligence and Data science at the School of Computer Science and Engineering at B.S. Abdur Rahman Crescent Institute of Science and Technology which is located at Vandalur, Chennai, India. His areas of interest lies in areas like Machine Learning, Artificial intelligence, and Data Science. His email id is abdulrahim82436@gmail.com .